

从零开始懂 AI4S

蛋白质结构预测与设计

——一本给高中生也能读懂的 40 页通识长文

散步

“...the native conformation is determined by the totality of interatomic interactions and hence by the amino acid sequence, in a given environment.”

(天然构象由原子间相互作用的整体所决定，因而也就由氨基酸序列所决定。)

——克里斯蒂安·安芬森 (Christian Anfinsen, 诺贝尔化学奖 1972)

“若蛋白质靠随机试遍所有构象来完成折叠，所需时间将超过宇宙的年龄——但它实际上只用毫秒到秒。”

——赛勒斯·莱文索尔 (Cyrus Levinthal, 1969) 提出的莱文索尔悖论

目录

第一部分 第一篇·先修：世界是由什么做的	5
1 第 0 章 开场：我们要解决什么问题	5
1.1 请先忘掉你已经知道的一切	5
1.2 一句话问题	5
1.3 为什么要关心这件事	5
1.4 全书地图	5
2 第 1 章 原子、分子与“形状”	6
2.1 原子：不可分割的小球（先这么理解）	6
2.2 分子：搭好的乐高	6
2.3 分子的“形状”：这是全文最重要的直觉	6
2.4 怎么描述“一个点在哪里”：坐标	6
2.5 怎么算两点之间有多远：勾股定理的升级	7
3 第 2 章 能量与稳定：为什么球会往山下滚	7
3.1 世界的懒惰法则	7

目录	2
3.2 分子之间的“喜欢”与“讨厌”	8
3.3 温度：让一切抖动起来	8
4 第 3 章 蛋白质：生命的乐高	9
4.1 氨基酸：20 个字母	9
4.2 从珠子到链：一级结构	9
4.3 二级结构：局部会自发形成两种“标准件”	9
4.4 三级结构：真正的难题	10
4.5 功能由形状决定：钥匙与锁	10
4.6 中心法则（通俗版）：蛋白质是从哪里来的	10
第二部分 第二篇 · 折叠问题：一个困扰了半个世纪的难题	11
5 第 4 章 折叠问题与两大悖论	11
5.1 正式提出问题	11
5.2 第一个悖论（其实是好消息）：Anfinsen 法则	11
5.3 第二个悖论（坏消息）：莱文索尔悖论	12
5.4 悖论的解答：折叠漏斗	12
5.5 第三个现实困难：数据鸿沟	13
5.6 第四个困难：孤儿蛋白	14
5.7 从“预测”到“设计”：反向问题	14
第三部分 第三篇 · 两件工具：三维几何与人工智能	15
6 第 5 章 几何工具箱：距离、旋转，与“什么变了什么没变”	15
6.1 为什么 AI 必须懂几何	15
6.2 距离矩阵：把“两两距离”列成一张表	16
6.3 三角不等式：合法性的第一道关卡	16
6.4 平移与旋转：什么变了，什么没变	17
6.5 两个关键词：不变性与等变性	17
7 第 6 章 人工智能入门：模型是怎么“学”的	18
7.1 模型就是一条公式	18
7.2 损失函数：给模型打分	18
7.3 梯度下降：蒙着眼睛下山	19
7.4 神经网络：把很多条公式叠起来	19
7.5 表示 (Representation)：把东西变成一串数字	20
7.6 注意力机制：学会“该看哪里”	20
7.7 训练 vs 推理：两个不同的阶段	21
第四部分 第四篇 · 攻克折叠问题：从老办法到 AlphaFold2	21

8 第 7 章 老办法为什么不够用	22
8.1 思路一：抄作业（同源建模）	22
8.2 思路二：直接模拟物理（分子动力学）	22
8.3 思路三：搜索（Rosetta）	23
8.4 思路四：从进化里找线索（MSA 与共进化）——最重要的思路	23
8.5 AlphaFold1：深度学习的第一次冲击	24
9 第 8 章 AlphaFold2 逐层拆解	25
9.1 总览：输入是什么，输出是什么	25
9.2 双轨表示：两条信息公路	25
9.3 Evoformer 的核心：三角注意力	26
9.4 结构模块：让网络直接“画”出坐标	26
9.5 FAPE 损失：在“各自的视角”下比较误差	27
9.6 Recycling：做完再检查几遍	27
9.7 置信度：pLDDT 与 PAE	27
第五部分 第五篇 · 从预测到设计：生成范式登场	28
10 第 9 章 扩散模型入门：把照片变成雪花，再学会倒放	29
10.1 什么是“生成”	29
10.2 扩散的第一步：把一张照片变成雪花	29
10.3 用公式描述加噪	29
10.4 扩散的第二步：学会“倒放”	30
10.5 生成：从纯噪声开始，一步步去噪	30
10.6 先算一个小例子（二维版）	31
10.7 为什么扩散比“一步一步往下猜”更好	31
11 第 10 章 AlphaFold3 与设计闭环：无中生有造蛋白	31
11.1 AlphaFold3：用扩散取代结构模块	32
11.2 反向折叠：ProteinMPNN	32
11.3 从头设计：RFdiffusion	33
11.4 难点：旋转不能像坐标那样直接加噪	33
11.5 设计闭环：三步走 + 一道筛子	33
第六部分 第六篇 · 动手：把全部数学手算一遍	35
12 第 11 章 三个点，从头算到尾	35
12.1 搭好玩具模型	35
12.2 第一步：算距离矩阵，检查它“合法”吗	35
12.3 第二步：旋转一下，看看什么变了、什么没变	36
12.4 第三步：在这个坐标上做一次扩散去噪	37

目录	4
13 第 12 章 动手写代码：一个等变去噪器	38
13.1 伪代码：先看流程	38
13.2 可运行的 PyTorch 参考实现	38
13.3 逐行讲解：这段代码在干什么	40
第七部分 第七篇 · 全景、避坑与附录	41
14 第 13 章 全景对比、常见误解与避坑指南	41
14.1 代际演进对比表	41
14.2 常见误解 Q&A	41
14.3 工程落地避坑指南	43
14.4 这个领域还剩下的开放问题	43
14.5 论文与评测导航	44
A 附录 A 阶梯 0 → 1 → 2：给想深入的读者	44
A.1 阶梯 0：三句话的直觉	44
A.2 阶梯 1：低维手算的闭环	44
A.3 阶梯 2：严格数学表述	44
B 附录 B 名词速查表	46
C 附录 C 自测题与答案	47

第一部分 第一篇·先修：世界是由什么做的

1 第 0 章 开场：我们要解决什么问题

1.1 请先忘掉你已经知道的一切

在开始之前，我们先做一个约定：这篇文章假设你只读到高中，没有学生物竞赛，没有学过大学化学，也没有写过代码。所有看起来“理所当然”的词——原子、分子、能量、蛋白质、神经网络、扩散——我们都会从你每天能看见的生活经验出发重新讲一遍。

如果你已经懂其中某一块，直接跳过去就好；文中有橙色“生活类比”盒子、黄色“想一想”盒子、青色“练习”盒子，以及紫色“进阶”盒子。紫色盒子是给想深入的读者的，第一次读可以直接跳过，不影响主线。

1.2 一句话问题

这篇文章要回答两个问题：

1. **预测 (Prediction)**：给我一串氨基酸的排列顺序，能不能算出它最终会折成什么三维形状？
2. **设计 (Design)**：反过来，我想要一个能“抓住”病毒的蛋白质，能不能直接把它形状画出来、再反推出该用怎样的氨基酸顺序？

第一个问题在 2020 年被 AlphaFold2 基本解决，轰动整个科学界；第二个问题在 2022–2023 年被 ProteinMPNN 与 RFdiffusion 变成工程问题。它们合起来，就是 AI for Science（用 AI 做科学发现，简称 **AI4S**）最漂亮的一场胜仗。

1.3 为什么要关心这件事

生活类比：钥匙与锁

蛋白质就像一把把形状各异的小钥匙，人体里的“锁”（比如病毒表面的蛋白、DNA、其他分子）只认特定形状的钥匙。**药物设计本质上就是配钥匙**：想让病毒失效？造一把能卡住它、让它动不了的钥匙。想让某个化学反应加速？造一把恰好能“掰弯”反应物的钥匙（这就是酶）。能随心所欲地配钥匙，就意味着能造新药、新酶、新材料。

1.4 全书地图

1. **第一篇 (第 1–3 章)**：补最底层的常识——原子怎么组成分子、能量怎么决定形状、蛋白质到底是什么。目标是让你能“在脑子里看见”一个分子。
2. **第二篇 (第 4 章)**：正式提出折叠问题，以及两个决定命运的悖论。
3. **第三篇 (第 5–6 章)**：补两件工具——三维几何（距离、旋转）与人工智能（神经网络、注意力）。
4. **第四篇 (第 7–8 章)**：看看老办法为什么不行，以及 AlphaFold2 怎么一举解决。

5. 第五篇（第 9–10 章）：从“预测”跃迁到“设计”——扩散模型与 RFDiffusion。
6. 第六篇（第 11–12 章）：动手。用三个点手算一遍全部数学，再读一份能跑的代码。
7. 第七篇（第 13 章 + 附录）：全景对比、避坑、名词表、自测题，以及给进阶读者的严格数学。

2 第 1 章 原子、分子与“形状”

2.1 原子：不可分割的小球（先这么理解）

两千多年前人们就在问：把一块金子一直切下去，切到不能再切，剩下的是什么？答案就是**原子**。在本文里，你完全可以把原子想象成一个个**小硬球**——这个图像不精确（真实原子更像一团“电子云”），但对理解蛋白质已经够用了。

自然界里有大约 90 多种天然原子，但**构成生命体的主要只有 6 种**：碳（C）、氢（H）、氧（O）、氮（N）、磷（P）、硫（S）。你可以先记住前四个，它们是蛋白质的主角。

生活类比：乐高积木

原子就像乐高积木的**颗粒**。不同种类的积木，凸起的“连接点”数量不同。碳原子有 **4 个**连接点，氧原子有 2 个，氢原子只有 1 个。所谓**化学键**，就是两个原子各拿出一个连接点“咔哒”扣在一起。因为碳能同时连 4 个伙伴，它就能搭出枝杈、长链、环——这正是生命分子如此复杂多样的原因。

2.2 分子：搭好的乐高

几个原子扣在一起，就构成了**分子**。水就是一个氧原子连两个氢原子（ H_2O ）；我们呼出的二氧化碳是一个碳连两个氧（ CO_2 ）。

2.3 分子的“形状”：这是全文最重要的直觉

关键来了：**分子不是画在纸上的平面图案，而是立体的**。两个原子之间的距离（键长）和三个原子之间的夹角（键角）都是**基本固定**的——就像乐高积木的连接杆长度固定一样。

所以，一旦你知道一个分子里有哪些原子、谁和谁相连，它的**大致形状就被决定了**，只剩下少数几个“可以旋转的关节”还能动。

生活类比：关节骨架

把一个分子想成**一串用吸管和珠子穿起来的手工模型**：吸管（化学键）长度固定，珠子（原子）固定在吸管两端，但吸管与吸管相连的**关节可以转动**。整串模型能摆出很多姿势——团成一团、拉成一条、绕成螺旋——但**无论怎么摆，吸管的长度和关节的位置关系不能变**。这就是分子“能变形但不能乱变”的原因。

2.4 怎么描述“一个点在哪里”：坐标

现在我们需要一个数学工具来描述形状。假设在一间屋子里有一只蚊子，你怎么告诉别人它 **exact** 的位置？

你会说：以墙角为原点，沿着地面的两条墙根各量一个距离，再加上离地面的高度。这三个数字就是**坐标** (x, y, z) 。

坐标：用三个数 (x, y, z) 描述空间中一个点的位置。

原点：我们规定好的“零点”，就像地图上的天安门。

坐标轴：从原点出发的三条互相垂直的方向，分别记为 x 轴、 y 轴、 z 轴。

一个分子有 N 个原子，就对应 N 个坐标点。所谓“蛋白质的结构”，本质上就是一张写着几百到几千个 (x, y, z) 的表格——这就是 AI 最终要预测出来的东西。

2.5 怎么算两点之间有多远：勾股定理的升级

初中我们学过：直角三角形中，斜边的平方等于两条直角边的平方之和 ($a^2 + b^2 = c^2$)。把它翻译成坐标语言，平面上两点 (x_1, y_1) 与 (x_2, y_2) 的距离就是

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. \quad (1)$$

到了三维，只不过多加上“高度差”这一项：

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}. \quad (2)$$

练习 1.1：手算两点距离

已知原子 A 在 $(1, 2, 3)$ ，原子 B 在 $(4, 6, 3)$ （单位可以想成“埃”，即一亿分之一厘米）。求它们的距离。

解：逐项相减并平方：

$$\begin{aligned}(x_1 - x_2)^2 &= (1 - 4)^2 = (-3)^2 = 9, \\(y_1 - y_2)^2 &= (2 - 6)^2 = (-4)^2 = 16, \\(z_1 - z_2)^2 &= (3 - 3)^2 = 0.\end{aligned}$$

相加得 $9 + 16 + 0 = 25$ ，开平方得 $d = \sqrt{25} = 5$ 。

答案：距离为 5。注意这是个 3-4-5 直角三角形——后面我们还会反复遇到它。

想一想

如果两点在同一高度 (z 相同)，三维距离公式会退化成什么？——没错，就是二维的勾股定理。这说明三维公式只是二维公式的自然推广，**不需要死记**。

3 第2章 能量与稳定：为什么球会往山下滚

3.1 世界的懒惰法则

你一定见过：球会自己滚下山坡，水会往低处流，拉开的橡皮筋松手会弹回去。这些现象背后是同一条法则——

能量最低原理：一个系统会自动趋向“能量最低”的状态，因为那样的状态最稳定。

这里的“能量”可以先粗略理解为**势能**：位置越高、被拉得越紧，势能越大；系统总是想把它降下来。

生活类比：坐在椅子上

你站着比坐着累，因为站着时你的重心更高、肌肉要持续对抗重力。坐下来最省力，也最“稳定”——这就是能量最低。分子也一样：它会自动调整自己的形状，去找到那个“最省力”的姿势。

3.2 分子之间的“喜欢”与“讨厌”

分子里的原子并不是各过各的，它们之间存在**相互作用**：

- **吸引**：带正电和带负电的部分会互相靠近（像磁铁的异性相吸）；
- **排斥**：两个原子靠得太近时，它们的电子云会互相排斥（像你硬把两块同极磁铁按在一起）。

一个分子最终的形状，就是在这无数个“喜欢”与“讨厌”之间找到的一个最舒服的折中——既让该靠近的靠近，又不让任何两个原子挤在一起。

生活类比：地铁里挤人

想象早高峰的地铁车厢：每个人都想离别人远一点（排斥），但又被车厢边界和同伴拉着不能走太远（吸引）。最终大家会站成一个“刚刚好”的分布——这就是分子在能量最低时的样子。

3.3 温度：让一切抖动起来

上面说的“稳定姿势”是在**绝对零度**下的理想情况。现实世界里，分子一直在被周围的水分子撞击，**不停地抖动**。温度越高，抖得越厉害。

这意味着：蛋白质并不是一个僵硬的雕塑，而是一直在**微微颤动**的。温度太高时，抖动会剧烈到把整个结构抖散——这就是**蛋白质变性**（煮鸡蛋时蛋清从透明变成白色固体，就是蛋清蛋白被加热抖散后黏成一团）。

本篇小结（你已会什么）

- 原子是小硬球，靠“连接点”（化学键）拼成分子，碳有 4 个连接点；
- 分子是立体的，键长键角基本固定，只有“关节”能转动；
- 用 (x, y, z) 三个数描述一个原子的位置，用勾股定理的三维版本算距离；
- 系统总是趋向能量最低的状态，分子的最终形状是“吸引与排斥”折中的结果；
- 温度让分子不停抖动，太高会把结构抖散。

4 第3章 蛋白质：生命的乐高

4.1 氨基酸：20 个字母

蛋白质是由一种叫**氨基酸**的小分子串起来的长链。自然界常用的氨基酸有 **20 种**，你可以把它们想象成 **20 种不同颜色、不同脾气的小珠子**：

- 有的珠子**喜欢水**（亲水），愿意待在蛋白质表面，跟周围的水分子做朋友；
- 有的珠子**讨厌水**（疏水），会拼命往里面躲，藏在蛋白质的“内核”里；
- 有的珠子带**正电或负电**，会和带相反电的珠子互相吸引；
- 有的珠子**很僵硬**（比如脯氨酸），会让链条拐一个硬弯；
- 有的珠子**很灵活**（比如甘氨酸），让链条能随意扭动。

生活类比：项链与字母

如果蛋白质是一句话，那 20 种氨基酸就是 **20 个字母**。字母的**排列顺序**决定这句话的意思；氨基酸的排列顺序决定这个蛋白质最终折成什么形状、能干什么活。

4.2 从珠子到链：一级结构

氨基酸之间通过一种叫**肽键**的连接方式，一个接一个串成一条长链。这条链上氨基酸的**排列顺序**，就叫蛋白质的一级结构（**Primary Structure**）。

一级结构：氨基酸的排列顺序（一维的“字母串”）。

二级结构：局部段落的规则形状（ α 螺旋、 β 折叠）。

三级结构：整条链折叠成的完整三维形状。

四级结构：多条链拼在一起形成的复合体。

4.3 二级结构：局部会自发形成两种“标准件”

你可能以为一条长链会乱成一团麻。其实不然——由于肽键本身的几何限制，链条的局部段落最常自发形成两种规则形状：

- **α 螺旋**（ α -helix）：像一根**电话线或弹簧**，链条绕着自己盘成螺旋，靠一圈圈之间的氢键固定。
- **β 折叠**（ β -sheet）：像**百叶窗或折扇**，几段链条并排躺在一起，也靠氢键互相拉住。

这两种“标准件”是 1951 年 Pauling 等人提出来的，是结构生物学的地基。你可以这样记：**螺旋是盘起来的，折叠是摊开的**。

生活类比：编织

一条毛线如果只是随便扔，会乱糟糟；但如果织毛衣，它会自然形成“平针”和“罗纹”两种基本花样。蛋白质的二级结构就相当于毛线的这两种基本织法，而整件毛衣的款式（三级结构）则由这些花样怎么拼接决定。

4.4 三级结构：真正的难题

三级结构就是：这条由螺旋、折叠和无规则卷曲拼起来的链，最终在三维空间里怎么摆放。**这才是本文要攻的难关**——因为二级结构基本是“局部规则”，而三级结构是“全局安排”，可能性多到爆炸。

4.5 功能由形状决定：钥匙与锁

为什么形状这么重要？因为蛋白质是**靠形状来工作的**。

- **酶**：表面有一个形状精巧的“凹槽”（活性位点），只有特定形状分子能卡进去；卡进去之后，酶会“掰弯”它，让某个化学反应瞬间变得容易发生。这就是生命体内几乎所有化学反应的加速方式。
- **抗体**：顶端有一个形状可变的“夹子”，能牢牢夹住病毒表面的特定部位，让它失去感染力。
- **血红蛋白**：形状能“呼吸”——在肺部结合氧气时是一种形状，到组织里释放氧气时又变成另一种形状。

所以：**形状 = 功能**。形状错了，功能就废了（比如镰刀型贫血，只是血红蛋白上一个氨基酸被换掉，形状改变，红细胞就变成镰刀形）。

4.6 中心法则（通俗版）：蛋白质是从哪里来的

最后补一块背景：蛋白质是怎么被制造出来的？

1. 细胞核里存着 **DNA**，它是一本用 4 种字母（A/T/C/G）写成的说明书；
2. 需要的时候，某一段 DNA 被抄写成 **RNA**（相当于一份临时复印件）；
3. 这份复印件被送到“工厂”（核糖体），工厂按上面的字母，**每 3 个字母对应 1 种氨基酸**，把氨基酸一个个串起来，就得到了蛋白质。

DNA → RNA → 蛋白质（氨基酸序列）→ 三维结构 → 生物功能

想一想

既然 3 个字母决定 1 种氨基酸，而字母有 4 种，那么不同的 3 字母组合有多少种？（提示： $4 \times 4 \times 4$ 。）这个数是 64，比氨基酸的 20 种多——说明**多个密码子会对应同一种氨基酸**，这叫密码子的简并性。这一点在第 10 章设计蛋白质时会再次出现。

本篇小结（你已会什么）

- 蛋白质由 20 种氨基酸串成，顺序就是“一级结构”；
- 局部会形成 α 螺旋与 β 折叠两种标准件（二级结构）；
- 整条链的三维摆法是三级结构，这是真正难的部分；
- 形状决定功能（钥匙与锁），形状错了功能就废；
- DNA $\rightarrow$ RNA $\rightarrow$ 蛋白质，3 个字母对应 1 种氨基酸。

下一篇，我们要问那个要命的问题：给定一个序列，它到底会折成什么形状？

第二部分 第二篇 · 折叠问题：一个困扰了半个世纪的难题

5 第 4 章 折叠问题与两大悖论

5.1 正式提出问题

现在我们可以把问题说清楚了：

折叠问题 (Protein Folding Problem)

输入：一条氨基酸序列（比如 “MKTFFVLLL...” 这样几百个字母）

输出：这条链上**每一个原子**在三维空间中的坐标 (x, y, z)

这个问题难在哪里？本章要讲两个“悖论”——它们一个告诉你**这事有希望**，另一个告诉你**这事难到离谱**。

5.2 第一个悖论（其实是好消息）：Anfinsen 法则

1950–60 年代，生物学家 Christian Anfinsen 做了一个漂亮的实验。他用一种叫**核糖核酸酶**的蛋白质，加入尿素和还原剂，把它**彻底“拆散”**——结构散开、变成一条软绵绵的乱链，功能完全丧失。

然后，他把**尿素和还原剂洗掉**。奇迹发生了：这条链**自己又折了回去**，恢复了原来的结构，也恢复了原来的功能。

这个实验说明了一件极其重要的事：

Anfinsen 法则：在给定环境下，蛋白质的天然构象（最终形状）完全由它的**氨基酸序列**决定。

换句话说：**序列本身就包含了结构的全部信息**，不需要额外的“说明书”。

生活类比：自动折叠的帐篷

有些帐篷你从袋子里倒出来，它“嘭”一下自己就撑开了。它之所以能自己撑开，是因为**支架的形状和连接方式（相当于序列）本身就编码了最终的立体形状**。Anfinsen 法则说的就是：蛋白质也是这样一种“自己会撑开”的结构。

为什么这是个好消息：它意味着“序列 → 结构”这条映射是**真实存在、且原则上可计算的**。既然信息就写在序列里，那么只要有足够聪明的模型和足够多的数据，AI 就有可能学会这条映射。这是整个 AlphaFold 事业的**理论许可证**。

5.3 第二个悖论（坏消息）：莱文索尔悖论

1969 年，分子生物学家 Cyrus Levinthal 提出了一个让人绝望的计算。

一条链上有许多可以转动的“关节”。为了估算，我们**极度保守**地假设：每个氨基酸只有 **3 种** 可能的姿势。那么一条 100 个氨基酸的链，总共有多少种可能的形状？

答案是 3 连乘 100 次：

$$3^{100} = 3 \times 3 \times \cdots \times 3 \text{ (共100 个)} \approx 5 \times 10^{47}. \quad (3)$$

这个数有多大？做个对比：**可观测宇宙中的原子总数大约是 10^{80} 个**，看起来还比它大；但关键在于——

生活类比：试遍所有组合要多久

假设有一种超级计算机，每秒能检查 10^{12} （一万亿）种形状。要试遍 5×10^{47} 种，需要

$$\frac{5 \times 10^{47}}{10^{12}} = 5 \times 10^{35} \text{ 秒} \approx 1.6 \times 10^{28} \text{ 年}.$$

而**宇宙的年龄只有约 1.4×10^{10} 年**（138 亿年）。也就是说，所需时间是宇宙年龄的 10^{18} 倍——一万万亿倍。

可是，真实的蛋白质折叠**只需要毫秒到秒**。这就是**莱文索尔悖论**：

如果蛋白质靠**随机试遍**所有构象来折叠，需要的时间远超宇宙年龄；
但它实际上只用毫秒到秒就折好了。

结论：折叠一定不是随机搜索，而是沿着某条“天然路径”进行的。

想一想

这个悖论听起来很绝望，但它其实也藏着希望。既然蛋白质“抄了近路”，说明**存在规律**——有规律，就意味着可以被学习。你能猜到是什么样的规律吗？（提示：回想第 2 章的“球往山下滚”。）

5.4 悖论的解答：折叠漏斗

答案是**能量景观漏斗（Folding Funnel）**。

想象一张地形图：横轴是“蛋白质所有可能的形状”（当然实际上是个超高维空间，这里画成二维），纵轴是“这个形状的能量”。如果地形是一片平坦的荒原，那蛋白质就得漫无目的地瞎逛（莱文索尔悖论的情形）。但真实的地形是一个**漏斗**：

- **漏斗上沿很宽**：大量高能量、乱七八糟的伸展形状；
- **越往下越窄**：形状越来越接近天然态；
- **漏斗底部**：能量最低的那一个（或几个）形状，就是**天然构象**。

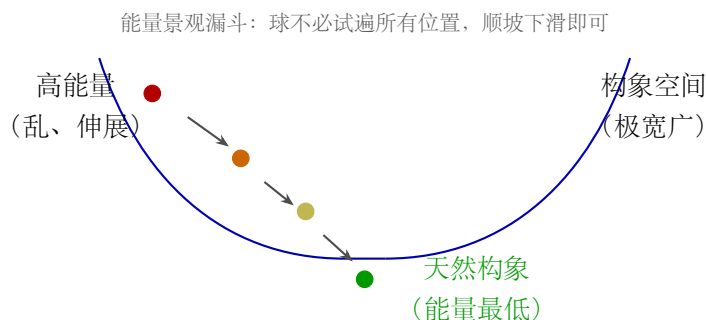


图 1: 折叠漏斗：蛋白质的能量景观不是一个平坦荒原，而是一个漏斗。球（蛋白质）不需要试遍所有位置，只要“顺坡往下滚”就能抵达底部的天然构象。这解释了为什么折叠既可**预测**（有规律）又**难搜索**（空间太大）。

生活类比：停车场找车位

如果停车场是一个巨大的平地、车位没有任何规律，你只能一格一格试（随机搜索，慢得要命）。但如果整个停车场是个斜坡，而你的目标车位在最低点，你只需松开刹车，车就会顺着坡自己滑向目标。折叠漏斗就是这样一个“斜坡”。

折叠漏斗同时解释了两件事，请务必记住这个双面结论：

1. **结构是可预测的**（好消息）：存在一条稳定的物理路径，说明序列与结构之间有规律可循，AI 学得会；
2. **搜索式方法很慢**（坏消息）：但如果你打算在计算机里模拟这个滚动过程（分子动力学），或者用程序去搜索这个空间（Rosetta），你就要面对那个天文数字的构象空间。

5.5 第三个现实困难：数据鸿沟

除了理论上的困难，还有一个极其现实的困难——**数据不对称**。

- **测序列非常便宜**：基因测序技术飞速发展，成本暴跌。公共数据库 UniProt 里的蛋白质序列已经达到**数亿条**。
- **测结构极其昂贵**：要看清一个蛋白质的三维结构，主要靠 X 射线晶体学、冷冻电镜（cryo-EM）或核磁共振（NMR），一个结构往往要几个月到几年、花费巨大，还可能根本做不出来。公共数据库 PDB 里只有**约二十万个结构**。

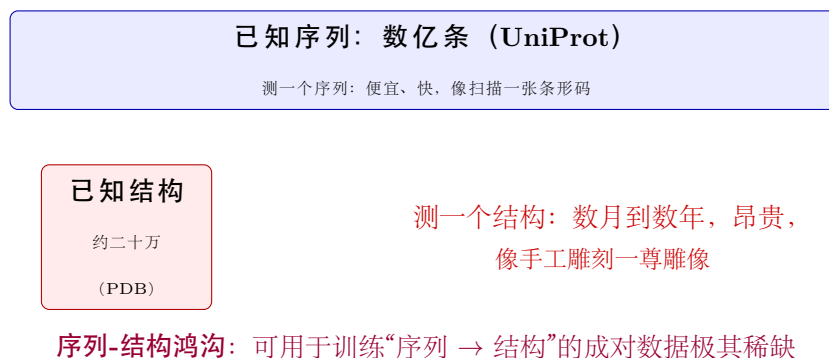


图 2: 序列-结构鸿沟: 序列数据以亿计且获取廉价, 结构数据仅约二十万且获取昂贵。这个不对称决定了: 任何依赖大量“序列-结构配对”的 AI 方法, 都必须想办法从海量无标注序列中榨取信息——这正是多序列比对 (MSA) 登场的理由。

想一想

如果你是 2018 年的研究者, 面对“序列多、结构少”的局面, 你会怎么利用那数亿条**没有结构标注**的序列? (提示: 进化会给你线索——功能重要的位置在千万年的演化中几乎不变。)

5.6 第四个困难: 孤儿蛋白

主流方法会从“进化线索”里提取信息 (具体做法见第 7 章的多序列比对 MSA)。但这里有个坑:

- **孤儿蛋白 (orphan protein):** 有些蛋白质在已知的序列库里**找不到任何亲戚**, 没有同源序列, 于是“进化线索”为空, 模型性能断崖式下跌。
- **人工从头设计的蛋白:** 这是更要命的一点——你要设计的蛋白质是自然界根本不存在的, 它当然没有任何亲戚! 也就是说, **恰恰在最需要 AI 帮忙的设计任务上**, 依赖进化线索的方法会集体失效。

5.7 从“预测”到“设计”: 反向问题

最后, 请记住: 即便预测做到了原子级精度, 生物学和药物研发真正需要的, 往往是**反过来**的问题。

- **正向 (预测):** 给我序列 → 告诉我它折成什么样。
- **反向折叠 (Inverse Folding):** 给我一个目标形状 → 告诉我用什么氨基酸序列能折出这个形状。
- **从头设计 (De novo Design):** 连形状都没有 → 直接**无中生有**地造出一个能结合某病毒、或能催化某反应的全新蛋白质。

2020 年之前, 后两件事近乎玄学; 2022–2023 年 ProteinMPNN 与 RFdiffusion 出现后, 它们才变成工程问题。

生活类比：看图说话 vs 按图施工

正向预测像“**看图说话**”：给你一串零件清单，告诉你组装完长什么样。反向折叠像“**按图施工**”：给你一张成品照片，让你反推该用哪些零件。从头设计则像“**自己当设计师**”：没有照片，只有一句“我要一个能夹住这块石头的夹子”，请你从零画图纸。难度显而易见是递增的。

本篇小结（你会什么）

- **Anfinsen 法则**：序列就包含了结构的全部信息 → 预测在原理上可行（好消息）；
- **莱文索尔悖论**：随机试遍 $3^{100} \approx 5 \times 10^{47}$ 种构象需要宇宙年龄的 10^{18} 倍时间，而真实折叠只要毫秒到秒 → 说明存在捷径；
- **折叠漏斗**：能量景观是漏斗而非荒原，顺坡下滑即可 → 既有规律（可学），又难搜索（空间太大）；
- **数据鸿沟**：序列数亿条 vs 结构约二十万条；**孤儿蛋白**与设计任务缺少进化线索；
- 真正的终极目标是**反向折叠与从头设计**，而不只是预测。

下一篇，我们先放下生物学，去补两件工具：三维几何（第 5 章）与人工智能（第 6 章）。

第三部分 第三篇 · 两件工具：三维几何与人工智能

6 第 5 章 几何工具箱：距离、旋转，与“什么变了什么没变”

6.1 为什么 AI 必须懂几何

回忆一下：蛋白质的结构 = 一堆原子的 (x, y, z) 坐标。所以 AI 输出的东西天然是一个**几何对象**。这就带来一个硬性要求：**AI 输出的坐标必须是一个“合法的三维结构”**

什么叫“合法”？举个最简单的例子：如果 AI 说原子 A 和 B 相距 3，原子 A 和 C 相距 4，那 B 和 C 的距离就**不能是 100**——因为 B 和 C 最多只能相距 $3 + 4 = 7$ 。这就是本章要讲的核心工具：**三角不等式**。

生活类比：三条边能组成三角形吗

给你三根木棒，长度分别是 3、4、100。你能把它们首尾相接围成一个三角形吗？不能——因为 3 和 4 两根接起来才 7，根本够不到 100 那么远。**三角不等式说的就是这件事**：任意两边之和必须大于第三边。

6.2 距离矩阵：把“两两距离”列成一张表

如果有 3 个原子，两两之间有一个距离，共 3×3 个（自己和自己是 0）。把它们排成一张表，就叫**距离矩阵**：

$$D = \begin{pmatrix} d_{11} & d_{12} & d_{13} \\ d_{21} & d_{22} & d_{23} \\ d_{31} & d_{32} & d_{33} \end{pmatrix} = \begin{pmatrix} 0 & d_{12} & d_{13} \\ d_{12} & 0 & d_{23} \\ d_{13} & d_{23} & 0 \end{pmatrix}. \quad (4)$$

注意它**沿对角线对称**（ $d_{12} = d_{21}$ ，因为 A 到 B 的距离等于 B 到 A 的距离），对角线上是 0。

练习 5.1：手算一个距离矩阵

三个原子在平面上： $\mathbf{r}_1 = (0, 0)$ ， $\mathbf{r}_2 = (3, 0)$ ， $\mathbf{r}_3 = (0, 4)$ 。请算出全部两两距离，写出距离矩阵。

解（用第 1 章的三维公式，这里 z 都是 0）：

$$\begin{aligned} d_{12} &= \sqrt{(0-3)^2 + (0-0)^2} = \sqrt{9} = 3, \\ d_{13} &= \sqrt{(0-0)^2 + (0-4)^2} = \sqrt{16} = 4, \\ d_{23} &= \sqrt{(3-0)^2 + (0-4)^2} = \sqrt{9+16} = \sqrt{25} = 5. \end{aligned}$$

所以

$$D = \begin{pmatrix} 0 & 3 & 4 \\ 3 & 0 & 5 \\ 4 & 5 & 0 \end{pmatrix}.$$

这就是著名的 3-4-5 直角三角形，本文后面会一直用它当玩具模型——因为它够简单，手算不费力，却麻雀虽小五脏俱全。

6.3 三角不等式：合法性的第一道关卡

对任意三个点，三条边必须同时满足两个条件（两条短边之和 $\geq$ 最长边；两边之差 $\leq$ 第三边）。合起来写就是

$$|d_{12} - d_{13}| \leq d_{23} \leq d_{12} + d_{13}. \quad (5)$$

代入我们的 3-4-5 例子：

$$|3 - 4| = 1 \leq 5 \leq 3 + 4 = 7. \quad \checkmark \quad (6)$$

成立！所以这一组距离**对应一个真实存在的结构**。

关键：为什么不合法的“距离”是个大问题

假设某个 AI 预测出的距离矩阵里， $d_{23} = 9$ 。那么 $9 > 3 + 4 = 7$ ，**违反三角不等式**。这意味着：**不存在任何一组真实的点，能满足这三个距离**——无论你怎么在空间中摆这三个原子，都摆不出来。

早期方法（AlphaFold1）正是先预测距离、再“硬凑”坐标。凑的时候如果距离矩阵本身不合法，那就像**拿着一张自相矛盾的地图找路**——再怎么努力也到不了，因为那个地方根本不存在。

练习 5.2：判断哪些距离矩阵是“合法”的

以下三组三边长度，哪些能组成真实三角形？

1. 3, 4, 5
2. 3, 4, 9
3. 5, 5, 5
4. 1, 1, 3

答案：(1) 合法 ($1 \leq 5 \leq 7$)；(2) 不合法 ($9 > 7$)；(3) 合法 (等边三角形, $0 \leq 5 \leq 10$)；(4) 不合法 ($3 > 1 + 1 = 2$)。

这提醒我们：AI 的输出必须过“合法性检查”，光看损失低是不够的。

6.4 平移与旋转：什么变了，什么没变

现在玩一个小游戏。把第 5.1 练习里的三角形拿起来，做两件事：

(a) **平移：**把它整体往右挪 5 格。新的坐标是 $\mathbf{r}'_1 = (5, 0)$, $\mathbf{r}'_2 = (8, 0)$, $\mathbf{r}'_3 = (5, 4)$ 。

(b) **旋转：**让它绕原点逆时针转 90° 。旋转的规则是：(x, y) 变成 $(-y, x)$ (这个规则你可以验证：把点 $(1, 0)$ 转 90° 应该到 $(0, 1)$ ，套公式得 $(-0, 1) = (0, 1)$ ，对)。于是

$$\mathbf{r}'_1 = (0, 0), \quad \mathbf{r}'_2 = (0, 3), \quad \mathbf{r}'_3 = (-4, 0). \quad (7)$$

现在重新算一遍两两距离：

$$d'_{12} = 3, \quad d'_{13} = 4, \quad d'_{23} = 5. \quad (8)$$

惊人又重要的发现

坐标变了，但两两之间的距离一个都没变。
也就是说：距离不依赖于你怎么摆放这个分子。

练习 5.3：自己验证一次

对旋转后的坐标 $\mathbf{r}'_2 = (0, 3)$ 与 $\mathbf{r}'_3 = (-4, 0)$ ，请手算 d'_{23} 。

$$d'_{23} = \sqrt{(0 - (-4))^2 + (3 - 0)^2} = \sqrt{16 + 9} = \sqrt{25} = 5.$$

和旋转前的 $d_{23} = 5$ 完全一样。你刚刚亲手验证了“距离在旋转下不变”

6.5 两个关键词：不变性与等变性

上面这个观察可以提炼成两个概念，它们是理解 AlphaFold 的钥匙（请务必弄懂，后面反复用到）：

不变性 (Invariance): 输入变了 (比如整个分子被旋转), 但输出的值完全不变。

例子: 两两距离 d_{12}, d_{13}, d_{23} ——旋转后还是 3, 4, 5。

等变性 (Equivariance): 输入变了, 输出跟着以同样的方式变。

例子: 坐标本身——你把分子转 90° , 坐标就整体转 90° 。

生活类比：看地图

把一张北京地图整体旋转 90° :

- 天安门和鸟巢之间的**实际距离**变了吗? 没有——这是**不变的**。
- 它们在**地图上的坐标**变了吗? 变了, 而且**跟着地图一起转了 90°** ——这是**等变的**。

一个好的模型应该像懂地图的人: 你转地图, 它的判断也跟着转, 而不会对转过的地图给出完全不同的答案。

想一想

为什么“等变性”对一个预测蛋白质结构的 AI 如此重要? 想一想: 如果你训练时用的分子都“头朝上”, 而测试时给你一个“横躺着”的同一个分子, 模型该给出什么答案?

答案是: 应该给出同一个结构, 只是跟着旋转了。如果一个不等变的模型看“横躺着”的分子就懵了, 那它必须把同一个分子的所有可能朝向都学一遍——这会浪费掉海量的训练数据。而蛋白质结构数据**极其稀缺** (第 4 章的数据鸿沟), 所以对 AlphaFold 来说, 等变性不是“锦上添花”, 而是**生死线**。

7 第 6 章 人工智能入门：模型是怎么“学”的

7.1 模型就是一条公式

别被“人工智能”四个字吓到。在最底层, 一个模型就是一条带参数的公式。

回忆初中的一次函数:

$$y = kx + b. \quad (9)$$

这里 x 是输入, y 是输出, 而 k 和 b 是**参数**。所谓“学习”(训练), 就是找到最合适的 k 和 b 。

生活类比：猜体重公式

你想根据身高 x 猜体重 y 。你猜 $y = kx + b$, 一开始随手设 $k = 1, b = 0$ (也就是“体重 = 身高”), 显然荒谬。于是你拿 100 个真实数据去试, 不断调整 k 和 b , 让猜出来的结果尽量接近真实体重。这个“不断调整”的过程, 就是机器学习。

7.2 损失函数：给模型打分

怎么知道猜得好不好? 需要一个**打分标准**, 叫**损失函数 (Loss Function)**。最常用的是“差的平方”:

$$\mathcal{L} = (y_{\text{预测}} - y_{\text{真实}})^2. \quad (10)$$

猜得越准，损失越小；损失为 0 就是完全猜对。

训练的目标就是：让损失尽可能小。

7.3 梯度下降：蒙着眼睛下山

有了打分标准，怎么调整参数？最经典的办法叫**梯度下降**（Gradient Descent）。

生活类比：蒙眼下山

想象你被蒙上眼睛放在一座山上，目标是走到山谷最低点。你看不见路，但脚下能感觉到**坡度**。策略很简单：

1. 用脚感受哪个方向是“下坡”；
2. 朝那个方向迈一小步；
3. 重复，直到感觉四面都是上坡（你已经到谷底了）。

这里的“坡度”在数学上就是**梯度**，而这个策略就是**梯度下降**。

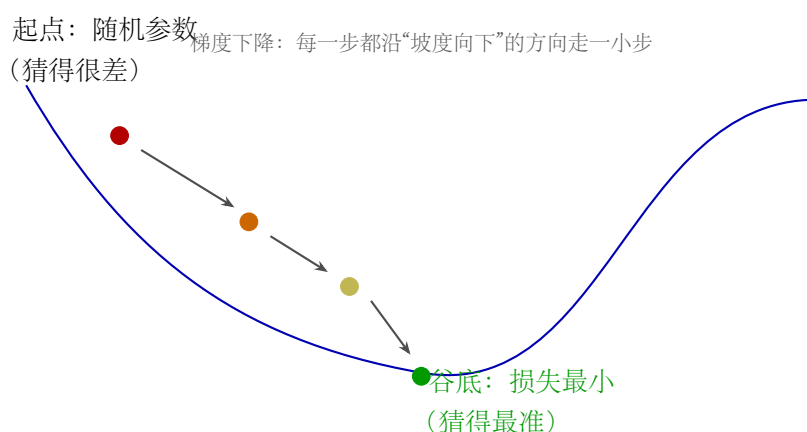


图 3：梯度下降示意：横轴是参数 k （或 b ），纵轴是损失 $\mathcal{L}$ 。从随机起点出发，每一步沿坡度向下的方向挪一小步，最终抵达损失最小的谷底。“步子多大”叫**学习率**，步子太大可能越过谷底，太小则走得太慢。

7.4 神经网络：把很多条公式叠起来

一次函数太简单了（只能画直线）。怎么表达更复杂的规律？答案是**叠加**：

1. 先对输入做很多个“ $kx + b$ ”的线性变换；
2. 每个结果再套一个**非线性函数**（比如 ReLU：负数变 0，正数不变）——这一步**至关重要**，没有它，叠多少层都还是一条直线；
3. 把上面两步重复很多次（这就叫“层”）；
4. 层数多了，就叫**深度学习**。

生活类比：工厂流水线

把神经网络想成一条**工厂流水线**：原料（输入数据）进来，经过第一道工序（第一层）粗加工，第二道工序再加工，……，最后一道工序产出成品（预测结果）。每一道工序的“机器参数”就是该层的参数，**训练就是同时调整所有工序的参数**，让最终成品的合格率最高。

7.5 表示 (Representation)：把东西变成一串数字

神经网络的输入必须是数字。所以我们要把各种东西**变成一串数字**，这串数字就叫**表示 (Representation)** 或**向量 / 嵌入 (Embedding)**。

- 一个原子 $\rightarrow$ 3 个数字 (x, y, z) ;
- 一个氨基酸 $\rightarrow$ 比如 128 个数字组成的向量，这 128 个数字是**学出来的**，语义相近的氨基酸向量会互相靠近；
- 一句话 $\rightarrow$ 每个词一个向量。

生活类比：用坐标表示人

怎么让计算机“理解”一个人？你可以用一串数字描述他：[身高, 体重, 年龄, 年收入, ...]。这串数字就是他的“表示”。两个人的数字串越接近，说明他们越相似。神经网络学的就是**怎么把事物变成好的数字串**。

7.6 注意力机制：学会“该看哪里”

这是现代 AI（包括 AlphaFold）的核心零件，我们用一个生活场景讲清楚。

生活类比：在人群里找朋友

你在火车站接人，站台上有一百个人。你不会**平均地**看这一百个人，而是**把注意力集中在像你朋友的那几个人身上**，其他的扫一眼就略过。这就是**注意力**。

具体分三步：

1. **查询 (Query, Q)**：你心里有个“我要找的人长什么样”的标准；
2. **键 (Key, K)**：站台上每个人都举着一块牌子，写着自己的特征（身高、衣服颜色……）；
3. **值 (Value, V)**：每个人身上携带的实际内容。

你拿自己的 **Q** 去和每个人的 **K** 比对，越匹配得分越高；然后用这些得分当**权重**，把所有人的 **V** 加权平均起来——结果就是“你真正关注到的内容”。

用公式写出来（先别怕，对照上面的故事读）：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V. \quad (11)$$

其中 softmax 的作用是把一堆得分变成**加起来等于 1** 的权重（像百分比）， $\sqrt{d}$ 只是个防止数值太大的缩放因子。

关键直觉：注意力让模型在处理某一个位置时，能**动态地**去“看”其他位置，并把看来的信息按重要程度混合进来。对蛋白质来说，这意味着：模型在判断第 37 号氨基酸的位置时，可以同时参考第 12 号、第 88 号……所有可能和它有关的氨基酸。

7.7 训练 vs 推理：两个不同的阶段

最后区分两个词，全文会反复用到：

训练 (Training)：拿大量“题目 + 答案”给模型做，做完对答案，用梯度下降调整参数。慢，但只做一次。

推理 (Inference)：模型训练好后，拿一道**没见过的新题目**让它给出答案。快，会做无数次。

生活类比：备考与考试

训练就像**备考**：刷大量真题（带答案），错了就翻书订正。推理就像**正式考试**：题目全是新的，你已经不能看书了，只能凭学到的本事作答。**考试题和刷题题最好是同一类，否则你就会“背答案”而不是“学方法”**——这就是过拟合，也是 AI 最重要的雷区之一。

本篇小结（你已会什么）

几何方面

- 距离矩阵记录两两距离，沿对角线对称；
- **三角不等式**是结构合法性的第一道关卡，违反它的距离矩阵对应不了任何真实结构；
- 平移旋转后：**距离不变（不变性）**，**坐标同步变换（等变性）**；
- 等变性让模型不必重复学习同一个分子的所有朝向，在数据稀缺时是生死线。

AI 方面

- 模型 = 带参数的公式；训练 = 调整参数让损失最小；
- 梯度下降 = 蒙眼下山，学习率是步长；
- 神经网络 = 多层“线性变换 + 非线性”叠加；
- 表示/嵌入 = 把事物变成一串数字；
- 注意力 = 用 Q 去匹配 K，得到权重，再加权平均 V；
- 训练（备考）与推理（考试）是两个阶段。

下一篇：有了这两件工具，我们终于可以去看科学家们是怎么一步步攻克折叠问题的了。

第四部分 第四篇 · 攻克折叠问题：从老办法到 AlphaFold2

8 第 7 章 老办法为什么不够用

有了前面的工具，我们来看看科学家们先后想出的几条路，以及每条路卡在哪里。这一段历史很重要，因为 AlphaFold2 的每一个设计，几乎都是在修补某一个老办法的漏洞。

8.1 思路一：抄作业（同源建模）

核心想法：演化是保守的。如果我知道蛋白质 *A* 的结构，而蛋白质 *B* 的序列和 *A* 很像，那 *B* 的结构大概率也和 *A* 差不多——直接“照着 *A* 套”就行了。这叫**同源建模（Homology Modeling）**。

什么时候灵：序列相似度高于 30 ~ 40% 时，相当可靠。

什么时候崩：一旦相似度跌到 20 ~ 30% 的“黄昏区（Twilight Zone）”，模板就不可靠了；而完全没有同源模板的全新折叠，它彻底束手无策。

生活类比：抄作业

抄作业很快，但前提是你手上有这道题（或极其相似的题）的答案。如果老师出了一道全新的题，你翻遍所有同学的作业本都找不到类似的，抄作业这条路就断了。同源建模就是这样——它只能处理“见过的题型”。

8.2 思路二：直接模拟物理（分子动力学）

核心想法：既然折叠是物理过程，那我就用牛顿力学把每个原子都算一遍：算出此刻每个原子受多少力，推出下一瞬间它们会移动到哪里，一小步一小步地积分下去。这叫**分子动力学（Molecular Dynamics, MD）**。

为什么崩（两个致命问题）：

1. **力场是近似的。**要算原子间的力，需要一套公式（叫“力场”）。但精确计算需要解量子力学方程，对成千上万个原子来说根本算不动，只能**用近似公式**。近似就意味着有误差，而误差会随着**成千上万步积分不断累积**，越算越偏。
2. **时间尺度根本不够。**真实折叠发生在微秒（ μs ）到秒的量级。而全原子 MD 为了稳定，每一步只能推进飞秒（ $1\text{ fs} = 10^{-15}\text{ 秒}$ ）量级。要模拟 1 毫秒，需要**十亿步**——即使动用超级计算机集群，也只能勉强摸到微秒级。

生活类比：用慢动作拍子弹

你想研究子弹穿过苹果的瞬间，于是用每秒 24 帧的普通摄像机去拍——结果什么也看不清，因为事情发生得太快了。MD 的问题反过来：**折叠这件事发生得太慢（相对计算机能算的步长而言）**，你要拍十亿帧才能看到结果。

8.3 思路三：搜索 (Rosetta)

核心想法：既然构象空间太大，那就别从头摸索，用“零件”拼。Rosetta 的做法是：

1. 把长链切成 3 个或 9 个氨基酸的小片段；
2. 从已知结构库里为每一小段找几套“现成的姿势”；
3. 用蒙特卡洛方法（带随机性的试错）不断替换片段姿势、打分、保留好的；
4. 重复成千上万次，生成上万条候选，挑最好的。

为什么崩：

- **打分函数（能量函数）不精确：**打分不准，搜得再久也找不到正确答案；
- **必须海量采样：**每个目标要生成上万条候选，慢，而且成功率很不稳定；
- **本质上它仍在那个天文数字的空间里搜索，**只是换了一种更聪明的搜索方式。

8.4 思路四：从进化里找线索 (MSA 与共进化) ——最重要的思路

这条路是 AlphaFold 真正的思想源头，值得多花点篇幅。

8.4.1 同源序列与多序列比对

同一个“工种”的蛋白质，往往存在于很多物种里。比如人类有一种负责运输氧气的蛋白，老鼠、鸡、鱼里也有功能类似的对应蛋白。它们的序列**不完全一样**（毕竟人和鱼差了很远），但**高度相似**——这些就叫**同源序列**。

把很多条同源序列**上下对齐排成一排**，让对应的位置对在一起，就得到了**多序列比对 (Multiple Sequence Alignment, MSA)**。

生活类比：多版本对照

想象《静夜思》在流传中产生了许多抄本，每个抄本都有几个字不一样。你把这些抄本并排摊开：

抄本 A：床前明月光 疑是地上霜

抄本 B：床前明月光 疑是地上雪

抄本 C：窗前明月光 疑是地上霜

你会发现：“前、明、月、光、疑、是、地、上”这些位置**几百年都没变**——说明它们**很重要**，改了会出问题；而“床/窗”“霜/雪”这些位置**经常变**——说明它们**不那么关键**。**不变 = 重要**，这就是 MSA 的第一条信息（保守性）。

8.4.2 共进化：会“互相配合着变”的位置

第二条信息更神奇，叫**共进化 (Co-evolution)**。

观察这样一个现象：在某些抄本里，“床”变成了“窗”，你本以为这是随机的；但你仔细一看，凡是“床”变成“窗”的抄本，另一个位置上的“霜”也一定变成了“雪”。两个位置总是一起变。

这说明了什么？说明**这两个位置有某种“配合关系”**——改了一个，另一个必须跟着改，否则整首诗就“坏掉”了。

对蛋白质来说，这种配合关系的**最常见原因就是：这两个氨基酸在三维结构里是贴在一起的（相互接触）**。

生活类比：双人舞

双人舞里，如果男舞者把手举高，女舞者就必须跟着转个身，否则两人会撞在一起。观众看不到他们的“配合规则”，但看几十场表演就会发现：**凡是男舞者举手的场次，女舞者一定转身**——于是推断出“这两个动作是耦合的”。

蛋白质同理：如果第 37 号氨基酸从“小个子”变成“大个子”，而所有发生这种变化的物种里，第 92 号氨基酸也都跟着从“大个子”变成“小个子”，那**第 37 号和第 92 号很可能在空间上是紧挨着的**——一个变大，另一个必须变小才装得下。

这就是全部的关键：MSA 里的共进化信号，能在完全不知道结构的情况下，告诉我们哪些**残基对在空间上是相互接触的**。它等于从海量（廉价的）序列数据里，免费挖出了（昂贵的）结构信息的线索。这正是破解“数据鸿沟”的办法。

想一想

回到第 4 章的那个问题：孤儿蛋白和人工从头设计的蛋白质，为什么会让依赖 MSA 的方法集体失效？（答案：它们没有任何“亲戚”，也就没有同源序列可供比对，共进化信号为空。）

8.5 AlphaFold1：深度学习的第一次冲击

2018 年 CASP13（两年一度的蛋白质结构预测“奥运会”），DeepMind 的 **AlphaFold1** 一鸣惊人夺冠。它的做法是：

1. 从 MSA 中算出共进化特征；
2. 用**卷积神经网络（CNN）**预测一张**残基-残基距离分布图**（也就是“每两个残基之间距离是多少”的概率分布）；
3. 再通过一个可微的势能函数，把这张距离图“优化”成三维坐标。

它证明了两件大事：共进化信号**极其有用**，而且深度学习**能把它榨干**。但它也暴露了三个致命崩点：

1. **几何不一致**：它预测的是距离图，而预测出来的距离矩阵不一定满足三角不等式（回顾第 5 章）。也就是说，它可能给出一张**自相矛盾的地图**，再怎么优化也凑不出合法的三维结构。
2. **MSA 依赖症**：孤儿蛋白、抗体 CDR 环、人工设计序列都没有同源序列，MSA 为空，性能断崖式下跌。
3. **只能预测，不能设计**：整条管线是单向的“序列 → 距离 → 结构”，没有任何机制能反过来做“结构 → 序列”。

承上启下

三个崩点正好对应 AlphaFold2 要解决的三个问题：

- 崩点 1（几何不一致）→ 用**三角注意力**把三角不等式写进架构（第 8 章）；
- 崩点 2（MSA 依赖）→ 后来由**蛋白质语言模型**路线（ESMFold）缓解（第 13 章）；
- 崩点 3（不能设计）→ 由 **ProteinMPNN + RFdiffusion** 解决（第 10 章）。

9 第 8 章 AlphaFold2 逐层拆解

2020 年 CASP14, AlphaFold2 以中位 GDT_TS 约 92 分（满分 100）的成绩，让整个结构生物学界惊呼“折叠问题在很大程度上被解决了”。这一章我们把它一层层拆开看。

9.1 总览：输入是什么，输出是什么

输入：

- 目标蛋白质的氨基酸序列（比如 200 个字母）；
- 它的 MSA（找到的同源序列，可能几十到几千条）。

输出：

- 每个原子的三维坐标 (x, y, z) ；
- 附带一个**置信度**打分（告诉你哪些部分靠谱、哪些不靠谱）。

整个网络可以看成三大块（图 4）：

1. **输入嵌入**：把序列和 MSA 变成数字；
2. **Evoformer（编码器）**：反复“通信”，把共进化线索和几何约束揉在一起，产出高质量的内部表示；
3. **结构模块（Structure Module）**：把内部表示直接解码成三维坐标。

9.2 双轨表示：两条信息公路

AlphaFold2 内部维护**两套表示**，你可以把它们想成两条并行的信息公路，彼此之间还有立交桥互通。

MSA 表示：形状是「序列数 $\times$ 残基数」。

它回答：这个位置在不同物种里是什么氨基酸？哪些位置总是一起变？（进化/共进化线索）

Pair 表示：形状是「残基数 $\times$ 残基数」。

它回答：第 i 个残基和第 j 个残基在空间上是什么关系？（几何关系）

生活类比：两份笔记

想象你在破案。**MSA** 表示是你的“嫌疑人关系网笔记”：谁和谁经常一起出现、谁的行踪和谁同步变化。**Pair** 表示是你的“案发现场地图笔记”：A 嫌疑人和 B 嫌疑人当时相距多远、朝向如何。

破案需要**两份笔记互相印证**：关系网告诉你“这两人可能有接触”，地图告诉你“他们实际不能接触到”。AlphaFold2 的 Evoformer 干的就是**让这两份笔记反复对账**，直到自洽。

9.3 Evoformer 的核心：三角注意力

Evoformer 由 48 层堆叠而成，其中最关键、也最体现 AlphaFold2 设计哲学的零件叫**三角更新 (Triangle Update)**。它的思想可以用一句话概括：

如果 i 和 j 有关系， i 和 k 有关系，那么 k 和 j 之间**也应该存在一致的关系**。
——“绕道走”得到的结果，应该和“直达”得到的结果对得上。

生活类比：三角航线

从北京直飞上海要 2 小时；如果北京 $\rightarrow$ 广州 $\rightarrow$ 上海，要 $2.5 + 2 = 4.5$ 小时。这两条路径的时间**必须满足三角不等式**：直飞不会比绕道更慢太多。

如果你拿到一张航班时刻表，发现“直飞 2 小时，但绕道只要 0.5 小时”，你立刻知道**这张表有错**——它不可能是真实的地理距离。**三角更新就是在检查并修正这类矛盾**：它强制 Pair 表示里的信息，始终像一个**真实三维空间里的距离关系**那样自洽。

这正是第 5 章三角不等式的**神经网络版本**：AlphaFold2 不再“先预测距离再硬凑坐标”，而是把**几何合法性直接写进了架构**——这是它相对 AlphaFold1 的第一个本质改进。

9.4 结构模块：让网络直接“画”出坐标

第二个本质改进在输出端。AlphaFold2 **不再输出距离图**，而是让结构模块**直接输出每个原子的三维坐标**。

它是怎么做到的？关键是一个叫 **IPA (Invariant Point Attention, 不变点注意力)** 的零件。它的思路极其漂亮，而且正好用上了第 5 章的等变性：

1. 给每个残基维护一个**局部坐标框架**（一个小坐标系，相当于“这个残基自己的上下左右前后”），这个框架属于 $SE(3)$ （三维旋转平移群）；
2. 在框架之间做注意力（回扣第 6 章的 Q/K/V）；
3. 计算注意力权重时，只用“**距离**”这类**不变量**（第 5 章：距离在旋转下不变），所以**权重是不变的**；
4. 输出时，用这些不变权重去加权**相对位移向量**（这些向量是等变的）；
5. **不变权重 $\times$ 等变位移 = 等变输出**。

生活类比：用“相对方位”指路

如果我说“学校在坐标 (120, 30)”，你换个城市就懵了。但如果我说“学校在你家**正东 500 米**”，无论你在哪个城市、面朝哪个方向，这句话都能用——因为**相对位移**是等变的、可迁移的。

IPA 正是这个道理：它不让网络去死记绝对坐标，而是让网络在**每个残基自己的局部框架里**思考“相对位置关系”。这样无论整个分子怎么旋转平移，网络的处理方式都完全一致——等变性被自动满足了。

9.5 FAPE 损失：在“各自的视角”下比较误差

既然直接输出坐标，损失函数怎么定？AlphaFold2 用了 **FAPE (Frame Aligned Point Error, 框架对齐点误差)**。

普通做法的问题：如果直接比较预测坐标和真实坐标的差距，那么“整个分子被摆放的位置和朝向”会严重影响误差——明明是同一个正确的结构，摆歪一点就被判为错。

FAPE 的做法：**不在全局坐标系里比，而是跑到每个残基自己的局部框架里去比**。在每个局部视角下算一遍预测原子和真实原子的距离，再把所有视角的结果平均起来。这样，**整体的旋转平移就被自动“消去”了**，误差只反映“结构本身对不对”。

生活类比：评判一座雕塑

要判断学生做的雕塑像不像真品，如果非要求它必须摆在展厅的同一个位置、朝同一个方向，那就太苛刻了。合理的方法是：**允许学生把雕塑挪动、旋转到最合适的角度，再看它像不像**。FAPE 就是这个“允许挪动旋转后再比较”的评分方式。

9.6 Recycling：做完再检查几遍

AlphaFold2 还有一个简单有效的技巧：把结构模块的输出**送回 Evoformer 再跑一遍**，如此循环若干次（叫 **Recycling, 循环回收**）。

生活类比：写作文后的修改

第一遍写完草稿，你再读一遍，发现某段的论据和结论对不上，于是修改；改完再读一遍，又发现另一处可以更通顺……**每一遍都在前一遍的基础上改进**。Recycling 就是让网络“多读几遍自己的草稿”。

9.7 置信度：pLDDT 与 PAE

最后，AlphaFold2 会给出**置信度**，这是它极其重要的工程价值——它告诉你**哪些预测能信，哪些不能**。

- **pLDDT (0–100, 逐残基)**：预测每个残基的位置有多可靠。> 90 非常可信；70 ~ 90 大体可信；50 ~ 70 要小心；< 50 基本不可信。
- **PAE (Predicted Aligned Error, 预测对齐误差)**：预测**两个结构域之间的相对位置**有多可靠。做复合物预测时，PAE 往往比 pLDDT 更关键。

重要提醒：pLDDT 低 $\neq$ 预测错了

很多初学者会误以为 pLDDT 低就是“模型算错了”。其实很多时候模型是对的——因为自然界存在一类叫**天然无序区（IDR）**的蛋白质片段，它们本来就没有固定形状，像一根软软的绳子一直在甩动。对这类区域，pLDDT 低正是**正确的判断**。

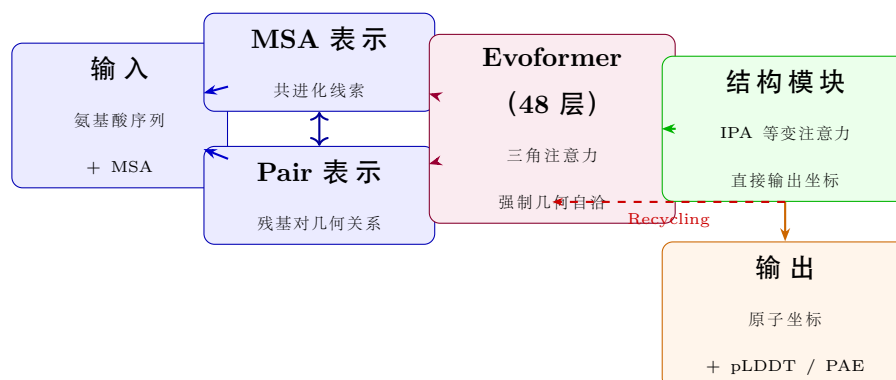


图 4: AlphaFold2 总览：序列与 MSA 进入双轨表示（MSA 表示 + Pair 表示），经 Evoformer 的三角注意力强制几何自洽，再由 SE(3) 等变的结构模块（IPA）直接输出原子坐标；Recycling 把输出送回反复精修，最后附带 pLDDT/PAE 置信度。

本篇小结（你会做什么）

老办法的漏洞

- 同源建模：没模板就崩；
- 分子动力学：力场近似 + 时间尺度不够（要算十亿步）；
- Rosetta：能量函数不精确 + 需海量采样；
- AlphaFold1：距离图可能违反三角不等式、MSA 依赖、不能设计。

AlphaFold2 的三根支柱

- **双轨表示**：MSA 表示（共进化）+ Pair 表示（几何），反复对账；
- **三角注意力**：把三角不等式写进架构，从源头保证几何合法；
- **IPA + FAPE**：用“不变权重 $\times$ 等变位移”实现 SE(3) 等变，在局部框架下算误差，直接输出坐标。

工程细节：Recycling 反复精修；pLDDT / PAE 给出置信度（注意 pLDDT 低常意味着天然无序区，不一定是错）。

下一篇：预测解决之后，更激动人心的反向问题来了——我们能不能**无中生有**地设计出自然界不存在的蛋白质？

第五部分 第五篇 · 从预测到设计：生成范式登场

10 第 9 章 扩散模型入门：把照片变成雪花，再学会倒放

到这一章，我们要引入一个全新的能力：**生成**。前面所有方法都是“给我输入，给我一个输出”；而生成要做的是“给我一片混沌，还我一个新东西”。

10.1 什么是“生成”

生成 (Generation)：从一个随机的起点（通常是一堆随机数，叫“噪声”）出发，一步步变成一个有意义的、此前不存在的新对象。

它可以是一张新照片、一段新视频、一个新分子，或——一个全新的蛋白质。

生活类比：雕塑家与石料

米开朗基罗说过，雕塑就是把“本来就藏在石头里的形象”凿出来。生成模型做的事很像：从一整块混沌的石料（纯噪声）出发，依据学到的“什么东西才算一个合理的对象”，一凿一凿地把多余的噪声去掉，最后露出一个新的作品。

注意这和“预测”的本质区别：预测是输入已决定答案（同一个序列永远折成同一个结构）；生成则是每次都能给出不一样的新答案（每次采样都能长出一个不同的新蛋白质）。

10.2 扩散的第一步：把一张照片变成雪花

扩散模型的想法，先从破坏讲起。

拿一张清晰的照片。我们做这样一件事：往照片上加一点点随机杂讯（噪声）。照片稍微变糊了。再加点，更糊。再加……加几百次之后，照片彻底变成了一片随机的雪花点，原来的内容一丝不剩。

这个过程叫前向过程（Forward Process），也叫加噪。

生活类比：反复复印

把一张画拿去复印，再把复印件拿去复印，如此反复几十次。每一代都会损失一点细节、多出一点杂点，到最后你得到的是一张几乎全是噪点的废纸。前向加噪就是这件事的高速版本。

10.3 用公式描述加噪

不要被公式吓到，它的意思就是“在原始内容和纯噪声之间按比例混合”。设原始结构（或图片）是 $\mathbf{x}_0$ ，随机噪声是 ϵ （每个数都从标准正态分布里随机抽），那么在“时间” t 加噪后的结果是

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (12)$$

其中 $\bar{\alpha}_t$ 是一个从 1 逐渐降到 0 的数（叫噪声调度）：

- $t = 0$ 时 $\bar{\alpha}_t = 1$ ：完全没有噪声， $\mathbf{x}_t = \mathbf{x}_0$ （原始清晰图）；

- $t = T$ 时 $\bar{\alpha}_t \approx 0$ ：全是噪声， $\mathbf{x}_t \approx \epsilon$ （纯雪花点）。

生活类比：调鸡尾酒

把“原始照片”想成一杯纯果汁，“噪声”想成白开水。 $\sqrt{\alpha_t}$ 就是果汁的比例， $\sqrt{1 - \alpha_t}$ 是水的比例。一开始全是果汁，越往后水越多，最后整杯都是水（纯噪声）。

10.4 扩散的第二步：学会“倒放”

真正的魔法在这里。我们训练一个神经网络，让它学会把加噪的过程倒过来：

给网络看一张加了噪的图，让它猜：当初加进去的噪声是什么？

猜得越准越好。因为——知道了噪声，就能把噪声减掉，图就变清晰了。

训练目标（损失函数）出人意料地简单，就是“猜噪声”的误差：

$$\mathcal{L} = \|\hat{\epsilon}_{\theta}(\mathbf{x}_t, t) - \epsilon\|^2. \quad (13)$$

这里 $\hat{\epsilon}_{\theta}$ 是网络猜的噪声， ϵ 是当初真正加进去的噪声， $\|\cdot\|^2$ 表示“逐项相减再平方求和”（就是第6章说的平方误差）。

生活类比：听写订正

老师在一段话里随机插入了一些噪音字，让你把插入的噪音字圈出来。你练得越多，圈得越准；圈出来之后把它们划掉，原话就恢复了。扩散模型练的就是“圈出噪音”的本事。

10.5 生成：从纯噪声开始，一步步去噪

训练好之后，要生成一个新东西，流程是：

1. 从一片纯随机的噪声 $\mathbf{x}_T$ 出发（什么都看不出来）；
2. 让网络猜“这里面有哪些是噪声”，减掉一点点；
3. 得到稍微干净一点的 $\mathbf{x}_{T-1}$ ，再让网络猜，再减掉一点；
4. 重复几十到几百步，噪声被逐级剥离，最终得到一个清晰的新对象 $\mathbf{x}_0$ 。

生活类比：洗照片

想象一张被严重噪点污染的老照片放在显影液里，你每隔几秒就擦掉一点点噪点。几十次之后，画面从一片雪花，慢慢浮现出轮廓，最后变成一张清晰的照片。而你事先并不知道会浮现出什么——这正是“生成”的魅力。

10.6 先算一个小例子（二维版）

为了让你确信这套东西真的能算，我们来做一个**二维的手算**。完整的三步演练放在第 11 章，这里先热个身。

取 $\bar{\alpha}_t = 0.8$ ，则 $\sqrt{\bar{\alpha}_t} \approx 0.8944$ ， $\sqrt{1 - \bar{\alpha}_t} \approx 0.4472$ 。设原始坐标 $\mathbf{x}_0 = (0, 4)$ ，随机噪声 $\epsilon = (0.2, -0.3)$ 。那么加噪后

$$\begin{aligned}\mathbf{x}_t &= 0.8944 \cdot (0, 4) + 0.4472 \cdot (0.2, -0.3) \\ &= (0 + 0.0894, 3.5776 - 0.1342) \\ &= (0.0894, 3.4435).\end{aligned}\tag{14}$$

如果网络猜噪声猜得不错，比如猜出 $\hat{\epsilon} = (0.18, -0.28)$ ，那损失就是

$$\mathcal{L} = (0.18 - 0.2)^2 + (-0.28 + 0.3)^2 = 0.0004 + 0.0004 = 0.0008,\tag{15}$$

非常小，说明猜得准。有了猜中的噪声，就能把 $\mathbf{x}_t$ “倒推”回干净坐标（第 11 章会完整推导）：结果大约是 $(0.01, 3.99)$ ，和真值 $(0, 4)$ 只差约 0.014。

你刚刚亲手验证了：**扩散模型在二维坐标上确实能 work**。而蛋白质设计要做的，只是把“二维坐标”换成“每个残基的三维坐标与朝向”。

10.7 为什么扩散比“一步一步往下猜”更好

你可能会问：为什么不直接让网络“一个残基一个残基地往下猜”（自回归），就像写作文一个字一个字写那样？

关键区别在于**误差会不会累积**：

- **自回归（逐帧/逐残基）**：把上一步的**预测**当成下一步的输入。一旦某一步偏了一点，这个偏差就会被喂回去，**越滚越大**——这正是第 7 章 AlphaFold1 与早期视频预测崩溃的根源。
- **扩散（全局去噪）**：每一步看到的都是**整个结构的当前状态**，不存在“把自己上一轮的猜测当输入”这个死循环。它更像**反复打磨同一块石料**，而不是**接龙讲故事**。

生活类比：接龙 vs 打磨

一群人玩故事接龙，每人接一句。如果第 3 个人理解偏了，后面所有人都会跟着偏，故事越跑越远——这是**自回归**。

而雕塑家雕一块石头，他每次都是**面对整块石头修一刀**。某一刀修歪了，下一刀可以直接补救，误差不会像接龙那样被放大——这是**扩散**。

11 第 10 章 AlphaFold3 与设计闭环：无中生有造蛋白

现在我们把“生成”这件新武器，装回到蛋白质问题上来。

11.1 AlphaFold3：用扩散取代结构模块

2024 年，DeepMind 发布 **AlphaFold3**（发表于 Nature）。它做了两处关键改动：

1. **Pairformer**：把 Evoformer 大幅简化（MSA 处理的比重降低）。理由是：共进化信号的利用方式，上一代已经验证得比较充分，可以更精简。
2. **扩散模块 (Diffusion Module)**：用去噪扩散直接生成原子坐标，取代 AlphaFold2 的 IPA 结构模块。

为什么要用扩散取代结构模块？有两个重要理由：

- **天然处理“多峰不确定性”** 有些蛋白质本来就不止一种形状（比如结合配体前是一个构象，结合后变成另一个）。让网络输出“唯一一个答案”，等于强行让它对多个合理答案取平均，结果往往是一个“四不像”。而扩散**每次采样都能给出一个完整、自洽的构象**，采样多次就能覆盖多种可能。
- **更容易扩展到复合物**。AlphaFold3 把预测对象从“蛋白质单体”扩展到了**蛋白质 + 配体（小分子药物） + 核酸（DNA/RNA） + 离子 + 修饰残基**的通用复合物。扩散框架对这种“多种类型对象混在一起”的场景更友好。

生活类比：画人像 vs 拍照片

让模型给一个“唯一答案”，就像要求画家把一对双胞胎画成一张脸——他只能取平均，结果是两个人都画得不像（多峰塌缩）。而扩散模型像是给这对双胞胎各拍一张照片，每张都清清楚楚，只是每次采样可能拍到不同的一位。

11.2 反向折叠：ProteinMPNN

预测问题解决后，第一个反向任务是**反向折叠 (Inverse Folding)**：

输入：一个目标骨架结构（三维坐标）

输出：一串能折成这个骨架的氨基酸序列

ProteinMPNN (2022) 就是干这个的。它是一个**等变图神经网络**，把骨架当成一张“图”（残基是点，空间上邻近的残基之间有边），然后一个残基一个残基地自回归地决定每个位置该放 20 种氨基酸里的哪一种。

一个反直觉但极其重要的事实：ProteinMPNN 的原生序列恢复率只有约 50%。这听起来很低，其实是**好事**——因为同一个骨架可以被大量不同的序列折出来（还记得第 3 章的密码子简并吗？这里是另一层“简并性”）。50% 的恢复率恰恰说明“设计空间非常广阔”，给了设计师大量可选方案。

生活类比：一题多解

一道几何题，标准答案只给了一种证法，但实际上有几十种证法。ProteinMPNN 的价值不是“复述标准答案”，而是**源源不断地给出新的、同样正确的解法**——这正是设计需要的。

11.3 从头设计：RFdiffusion

第二个反向任务更难：连骨架都没有，直接从噪声里“长”出一个全新的蛋白质骨架。这就是 **RFdiffusion (2023)**。

它的做法可以一句话概括：把 **RoseTTAFold**（另一个结构预测网络）改造成一个去噪扩散模型。具体来说：

- **训练时**：拿真实蛋白质结构，往它的坐标上加噪，训练网络去猜噪声（第 9 章的套路）；
- **推理时**：从一片纯噪声出发，让训练好的网络一步步去噪，最终“长”出一个全新的、合理的蛋白质骨架。

它支持多种生成模式：

- **无条件生成**：给我一个全新的蛋白质，什么都行；
- **对称性生成**：给我一个由 6 个相同单元拼成的环状结构；
- **条件生成 (Scaffolding)**：保留指定的功能位点不动，只在它周围“长出”支撑它的骨架——这在设计结合蛋白 (binder) 时极其有用。

11.4 难点：旋转不能像坐标那样直接加噪

这里有一个很微妙但很关键的技术点，值得单独讲。

对**平移**（位置， $\mathbb{R}^3$ ）加噪很容易：坐标是普通的三个数，直接加高斯噪声就行，就像第 9 章那样。

但每个残基除了“在哪里”，还有“朝哪个方向”（朝向，属于 $SO(3)$ ，即三维旋转群）。给“朝向”加噪就不能简单加随机数了。

为什么？因为**旋转必须始终是合法的旋转**。一个合法的旋转矩阵必须满足“每一列都是单位长度、且列与列互相垂直”这样的约束。你随便往旋转矩阵里加噪声，加完之后它就不再是合法的旋转了——就像你把地图上的“北”字随便挪一挪，指南针就废了。

生活类比：在地球表面上走直线

地球是个球面。你在北京朝正东走 100 公里，和在北京朝正东“在平面地图上”画一条 100 公里的直线，终点**不是同一个地方**——因为球面是弯的，不能当平面处理。

旋转的“空间” $SO(3)$ 也是一个**弯曲的空间（数学上叫流形）**。在弯曲空间里，“加噪声”必须**沿着弯曲的表面走**（比如：绕一个随机轴，转一个随噪声强度增大的随机角度），而不能像在平面上那样直接“加随机数”。

所以 RFdiffusion 的做法是：平移部分在普通欧氏空间加噪，旋转部分在 $SO(3)$ 这个弯曲空间上按它自己的规则加噪。两者合起来就是 $SE(3)$ （三维刚体运动群）。这是第 11 章和附录会进一步说明的地方。

11.5 设计闭环：三步走 + 一道筛子

把 RFdiffusion、ProteinMPNN 和 AlphaFold 串起来，就得到了当代蛋白质设计的**标准流水线**（图 5）：

1. **RFdiffusion 生成骨架**：从噪声里“长”出一个符合要求的三维骨架（比如能贴住病毒表面的某个位置）；
2. **ProteinMPNN 设计序列**：为这个骨架配一串能折成它的氨基酸序列（通常会设计很多条候选）；
3. **AlphaFold2/3 自洽性检验**：把设计出的序列丢回 AlphaFold 重新预测——如果它折回来的样子和设计目标高度吻合（误差小、置信度高），说明这条设计**自洽**，可信；
4. **湿实验验证**：只有通过了自洽性检验的少量候选，才值得送去真正做实验（合成基因、表达蛋白、测结构、测功能）。

生活类比：建筑设计 + 计算机模拟

建筑师画好图纸（RFdiffusion 骨架）；结构工程师选材料（ProteinMPNN 序列）；然后把图纸丢进计算机做结构仿真，看楼会不会塌（AlphaFold 自洽性检验）；仿真通过之后，才真正动工盖楼（湿实验）。

为什么自洽性检验这道筛子如此关键：湿实验又慢又贵（一个结构几个月到几年）。自洽性检验是纯计算机的，几秒到几分钟就能筛掉 99% 的不靠谱设计，把宝贵的实验资源留给最有希望的候选。

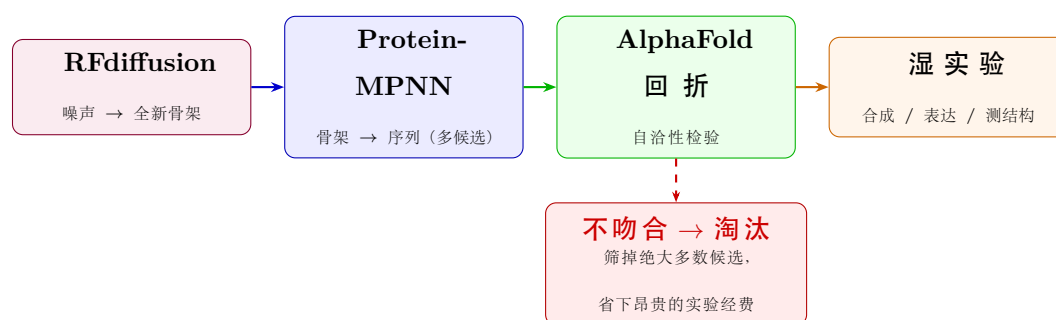


图 5: 蛋白质设计闭环：RFdiffusion 生成骨架 → ProteinMPNN 设计序列 → AlphaFold 回折做自洽性检验 → 通过的少量候选才送湿实验。这道“计算机筛子”把设计成功率提升了数量级，是从预测跃迁到设计的落地形态。

本篇小结（你会什么）

扩散模型

- 前向加噪：把清晰内容按比例与噪声混合，最终变成纯噪声；
- 训练目标极简：让网络猜“加进去的噪声是什么” 损失就是猜噪声的平方误差；
- 生成：从纯噪声出发，一步步减掉猜出的噪声；
- 相比自回归，扩散不把自己的猜测当输入，因此不累积误差，还能处理多峰不确定性。

从预测到设计

- AlphaFold3 用扩散模块取代结构模块，扩展到蛋白-配体-核酸复合物；
- ProteinMPNN 做反向折叠（结构 → 序列），50% 恢复率反而说明设计空间广阔；
- RFdiffusion 从噪声生成全新骨架；旋转属于弯曲空间 $SO(3)$ ，不能像坐标那样直接加噪；
- 标准闭环：生成骨架 → 设计序列 → AlphaFold 自治性检验 → 湿实验。

下一篇：光说不练假把式。我们用三个点，把全部数学从头手算一遍。

第六部分 第六篇 · 动手：把全部数学手算一遍

12 第 11 章 三个点，从头算到尾

前面十章讲了很多概念。这一章我们全部放下，只用三个点，把“几何合法性 → 等变性 → 扩散生成”这条主线一步不跳地手算一遍。

之所以选三个点，是因为：三个点能构成三角形，而三角形刚好是最小的、能体现三维结构全部难度的单元——它有距离、有角度、有刚性，还能旋转。

12.1 搭好玩具模型

我们把蛋白质简化成平面上的三个残基（所以是 $D = 2$ ），取一个最经典的位置——3-4-5 直角三角形：

$$\mathbf{r}_1 = (0, 0), \quad \mathbf{r}_2 = (3, 0), \quad \mathbf{r}_3 = (0, 4). \quad (16)$$

你可以随手在草稿纸上画出来： $\mathbf{r}_1$ 在原点， $\mathbf{r}_2$ 在它右边 3 格， $\mathbf{r}_3$ 在它上方 4 格。

12.2 第一步：算距离矩阵，检查它“合法”吗

(1a) 逐对算距离（用第 1 章的公式，这里 z 分量都是 0）：

$$d_{12} = \sqrt{(0-3)^2 + (0-0)^2} = \sqrt{9+0} = 3, \quad (17)$$

$$d_{13} = \sqrt{(0-0)^2 + (0-4)^2} = \sqrt{0+16} = 4, \quad (18)$$

$$d_{23} = \sqrt{(3-0)^2 + (0-4)^2} = \sqrt{9+16} = \sqrt{25} = 5. \quad (19)$$

(1b) 写成距离矩阵:

$$D = \begin{pmatrix} 0 & 3 & 4 \\ 3 & 0 & 5 \\ 4 & 5 & 0 \end{pmatrix}. \quad (20)$$

(1c) 检查三角不等式 (第 5 章):

$$|d_{12} - d_{13}| = |3 - 4| = 1 \leq d_{23} = 5 \leq d_{12} + d_{13} = 3 + 4 = 7. \quad \checkmark \quad (21)$$

结论: 这一组距离合法, 对应一个真实存在的结构。

反面例子 (关键)。假如某个模型预测出的距离矩阵是

$$D_{\text{bad}} = \begin{pmatrix} 0 & 3 & 4 \\ 3 & 0 & 9 \\ 4 & 9 & 0 \end{pmatrix}. \quad (22)$$

检查上界: $d_{23} = 9$, 但 $d_{12} + d_{13} = 7$, 而 $9 > 7$ ——违反三角不等式。

这意味着: **不存在任何三个点能满足这组距离**。你可以自己试试: 先在纸上点两个点相距 3, 再以第一个点为圆心画半径 4 的圆 (第三个点必须在这个圆上), 再以第二个点为圆心画半径 9 的圆。你会发现**这两个圆根本不相交**——因为两个圆的半径之和 ($3+9=12$) 虽然大于圆心距, 但半径之差 ($9-3=6$) 大于 4……总之第三个点无处安放。

这正是第 7 章 AlphaFold1 崩点 1 的微观复现: 先猜距离、再硬凑坐标, 凑出来的“距离”可能根本凑不出真实结构。

12.3 第二步: 旋转一下, 看看什么变了、什么没变

(2a) 平移: 把三角形整体往右挪 5 格 (每个点 x 加 5):

$$\mathbf{r}_1^{\text{移}} = (5, 0), \quad \mathbf{r}_2^{\text{移}} = (8, 0), \quad \mathbf{r}_3^{\text{移}} = (5, 4). \quad (23)$$

(2b) 旋转: 让它绕原点逆时针转 90° 。规则是 $(x, y) \mapsto (-y, x)$:

$$\mathbf{r}'_1 = (0, 0), \quad \mathbf{r}'_2 = (0, 3), \quad \mathbf{r}'_3 = (-4, 0). \quad (24)$$

为什么旋转 90° 的规则是 $(x, y) \rightarrow (-y, x)$?

拿一个最简单的点验证: 点 $(1, 0)$ 在 x 轴正方向上。把它逆时针转 90° , 应该跑到 y 轴正方向, 也就是 $(0, 1)$ 。套公式: $(-y, x) = (-0, 1) = (0, 1)$ 。✓再验一个: 点 $(0, 1)$ 转 90° 应该到 $(-1, 0)$, 套公式得 $(-1, 0)$ 。✓所以公式是对的。(学过三角函数的同学可以自己用旋转矩阵推一遍。)

(2c) 重算距离，一个一个来：

$$d'_{12} = \sqrt{(0-0)^2 + (0-3)^2} = \sqrt{9} = 3, \quad (25)$$

$$d'_{13} = \sqrt{(0-(-4))^2 + (0-0)^2} = \sqrt{16} = 4, \quad (26)$$

$$d'_{23} = \sqrt{(0-(-4))^2 + (3-0)^2} = \sqrt{16+9} = \sqrt{25} = 5. \quad (27)$$

(2d) 下结论：

- 坐标变了：从 $(0,0), (3,0), (0,4)$ 变成 $(0,0), (0,3), (-4,0)$ ——而且是跟着同一个规则一起变的，这就是等变性；
- 距离完全没变：还是 3, 4, 5 ——这就是不变性。

这就是第 8 章 IPA 与 FAPE 想让网络做到的事：你把分子转个角度，网络的输出也跟着转（等变），而它内部的判断（比如“这两个残基挨不挨着”）完全不变。

12.4 第三步：在这个坐标上做一次扩散去噪

最后，我们用第 9 章的扩散公式，在 $\mathbf{r}_3 = (0, 4)$ 这个点上完整走一遍加噪 → 猜噪声 → 反推。

(3a) 设定噪声调度。取 $\bar{\alpha}_t = 0.8$ ，于是

$$\sqrt{\bar{\alpha}_t} = \sqrt{0.8} \approx 0.8944, \quad \sqrt{1 - \bar{\alpha}_t} = \sqrt{0.2} \approx 0.4472. \quad (28)$$

(3b) 采样一个随机噪声。设 $\epsilon = (0.2, -0.3)$ （真实训练中这些数是从正态分布里随机抽的，这里我们直接给定，方便手算）。

(3c) 前向加噪，套公式 $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ ：

$$\begin{aligned} \mathbf{x}_t &= 0.8944 \cdot (0, 4) + 0.4472 \cdot (0.2, -0.3) \\ &= (0, 3.5776) + (0.0894, -0.1342) \\ &= (0.0894, 3.4435). \end{aligned} \quad (29)$$

(3d) 让网络猜噪声。设网络输出 $\hat{\epsilon} = (0.18, -0.28)$ （它猜得相当接近真值 $(0.2, -0.3)$ ）。

(3e) 算损失（第 6 章的平方误差）：

$$\begin{aligned} \mathcal{L} &= \|\hat{\epsilon} - \epsilon\|^2 \\ &= (0.18 - 0.2)^2 + (-0.28 - (-0.3))^2 \\ &= (-0.02)^2 + (0.02)^2 \\ &= 0.0004 + 0.0004 = 0.0008. \end{aligned} \quad (30)$$

损失很小，说明网络猜得准。

(3f) 由猜出的噪声反推干净坐标。思路很朴素：既然 $\mathbf{x}_t$ 是“真值 $\times 0.8944$ ”加上“噪声 $\times 0.4472$ ”得来的，那我就把噪声那部分减掉，再除以 0.8944：

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}}. \quad (31)$$

一步一步代入：

$$\begin{aligned}
 \hat{\mathbf{x}}_0 &= \frac{(0.0894, 3.4435) - 0.4472 \cdot (0.18, -0.28)}{0.8944} \\
 &= \frac{(0.0894, 3.4435) - (0.0805, -0.1252)}{0.8944} \\
 &= \frac{(0.0089, 3.5687)}{0.8944} \\
 &= (0.0100, 3.9900).
 \end{aligned} \tag{32}$$

(3g) 检查误差。真值是 $\mathbf{r}_3 = (0, 4)$ ，预测是 $(0.01, 3.99)$ ：

$$\text{误差} = \sqrt{(0.01 - 0)^2 + (3.99 - 4)^2} = \sqrt{0.0001 + 0.0001} \approx 0.0141. \tag{33}$$

误差只有约 0.014，而整个坐标的尺度是 4 左右——**相对误差不到千分之四**。

第 11 章小结：三步串成一条线

1. **几何合法性**：3-4-5 满足三角不等式 → 结构存在；改成 9 就不存在 → 这就是 AlphaFold1 崩掉的根源。
2. **等变与不变**：旋转后坐标整体跟着转（等变），距离一个没变（不变）→ 这就是 IPA 与 FAPE 的几何灵魂。
3. **扩散生成**：加噪 $(0, 4) \rightarrow (0.0894, 3.4435)$ ；猜噪声损失 0.0008；反推回 $(0.01, 3.99)$ ，误差 0.014 → 这就是 RFDiffusion 生成新骨架的最小版本。

请记住这个感觉：真实蛋白质只是把“二维、三个点”换成“三维、几千个原子”，但背后的数学完全一样。

13 第 12 章 动手写代码：一个等变去噪器

这一章我们把第 11 章的手算变成**能跑的代码**。即使你没写过 Python，跟着注释读也能明白每一行在干什么。

13.1 伪代码：先看流程

13.2 可运行的 PyTorch 参考实现

下面这份代码是**参考实现**，需要先安装 PyTorch (`pip install torch`) 才能运行。它做的事是：构造一个**二维的等变去噪器**——输入一堆带噪的二维坐标，输出每个点应该往哪边挪才能变干净。代码最后还会**验证第 11 章的等变性**：把输入整体转 90° ，看输出是不是也正好转了 90° 。

```

1  import torch, torch.nn as nn, torch.nn.functional as F
2
3  class EquivariantDenoiser(nn.Module):
4      """Input x: (B,N,2); output displacement: (B,N,2)."""
5      def __init__(self, hidden=64):

```

Algorithm 1 RFdiffusion 风格骨架生成：训练与采样**输入：**一批真实的蛋白质结构（每个都是一堆三维坐标），一个等变去噪网络 $\hat{\epsilon}_\theta$ **输出：**训练好的网络，可用于从噪声生成新骨架

```

1: // ————— 训练阶段（备考） —————
2: repeat
3:   取一个真实结构  $\mathbf{x}$ ，随机旋转平移一下做数据增广
4:   随机生成一个噪声  $\epsilon$ ，随机选一个时间步  $t$ 
5:   对平移加噪、对朝向在  $SO(3)$  上按弯曲空间的规则加噪
6:   加噪：  $\mathbf{x}_t \leftarrow \sqrt{\alpha_t} \mathbf{x} + \sqrt{1 - \alpha_t} \epsilon$ 
7:   让网络猜噪声：  $\hat{\epsilon} \leftarrow \hat{\epsilon}_\theta(\mathbf{x}_t, t)$ 
8:   算损失：  $\mathcal{L} \leftarrow \|\hat{\epsilon} - \epsilon\|^2$ 
9:   用梯度下降（第 6 章的蒙眼下山）调整网络参数
10: until 损失不再下降
11: // ————— 采样阶段（考试） —————
12: 从一团纯随机噪声  $\mathbf{x}_T$  出发
13: for  $t = T$  到 1 do
14:   让网络猜噪声，减掉一点点，得到更干净的  $\mathbf{x}_{t-1}$ 
15: end for

```

输出：得到一个全新的蛋白质骨架 $\mathbf{x}_0$

```

6     super().__init__()
7     # Small MLP: 3 features -> 1 scalar weight
8     self.net = nn.Sequential(
9         nn.Linear(3, hidden), nn.ReLU(),
10        nn.Linear(hidden, hidden), nn.ReLU(),
11        nn.Linear(hidden, 1))
12
13    def forward(self, x):          # x: (B,N,2)
14        # Step 1: relative displacements (equivariant)
15        rel = x.unsqueeze(2) - x.unsqueeze(1) # (B,N,N,2)
16        # Step 2: pairwise distances (rotation-invariant)
17        dist = rel.pow(2).sum(-1, keepdim=True).sqrt()
18        # Step 3: join displacement and distance features
19        feat = torch.cat([rel, dist], dim=-1) # (B,N,N,3)
20        # Step 4: predict invariant scalar weights
21        w = self.net(feat).squeeze(-1)        # (B,N,N)
22        w = w - w.mean(dim=1, keepdim=True)    # translation-invariant
23        # Step 5: invariant weight x equivariant displacement
24        return (w.unsqueeze(-1) * rel).sum(2) / x.size(1)
25
26    torch.manual_seed(0)
27    net = EquivariantDenoiser()
28    opt = torch.optim.Adam(net.parameters(), lr=1e-2)

```

```

29 B, N = 32, 8                                # 32 samples, 8 points each
30 base = torch.randn(B, N, 2) * 2.0           # synthetic clean structures
31
32 for step in range(150):                       # repeatedly predict noise
33     opt.zero_grad()
34     eps = torch.randn(B, N, 2)               # injected ground-truth noise
35     at = torch.tensor(0.8).sqrt().item()      # sqrt(alpha_bar_t)
36     xt = at * base + (1 - at**2).sqrt() * eps # add noise
37     loss = F.mse_loss(net(xt), eps)           # noise-prediction MSE
38     loss.backward()                          # compute gradients
39     opt.step()                               # one descent step
40
41 # Equivariance test: rotate input and output by 90 degrees
42 rot = torch.tensor([[0., -1.], [1., 0.]])
43 with torch.no_grad():
44     z = torch.randn(1, N, 2)
45     out = net(z)                             # original output
46     out_rot = net(z @ rot.T)                 # output after rotated input
47     expect = out @ rot.T                    # expected rotated output
48     print("equivariance error:", (out_rot - expect).abs().max().item())
49 print("final denoising loss:", loss.item())

```

Listing 1: 二维等变去噪器：训练 + 等变性检验

13.3 逐行讲解：这段代码在干什么

1. **第 3–13 行（定义网络）**：我们定义了一个小网络。它的输入是 3 个特征，输出 1 个“权重”。关键在第 19–25 行怎么用这个权重。
2. **第 16 行（相对位置差）**：对每个点 i ，算出它到所有点 j 的**相对位移** $\mathbf{r}_j - \mathbf{r}_i$ 。这是等变的——你把整个图形转一下，这些相对位移也跟着转。
3. **第 18 行（距离）**：算出两两距离。这是**不变的**（第 11 章第二步验证过：旋转后距离还是 3, 4, 5）。
4. **第 20 行（拼特征）**：把“等变的位移”和“不变的距离”拼在一起喂给小网络。
5. **第 22 行（输出权重）**：小网络给每一对点输出一个权重。因为这些权重是从“不变的距离”等信息算出来的，所以权重本身是不变的。
6. **第 23 行（减均值）**：减去权重的平均值，这一步保证网络对**平移**也不敏感。
7. **第 25 行（最关键的一行）**：**不变权重 $\times$ 等变位移 = 等变输出**。这正是第 8 章讲的 IPA 核心机制！代码里 `(w.unsqueeze(-1) * rel).sum(2)` 做的就是“用不变的权重去加权平均那些等变的位移”。
8. **第 33–41 行（训练循环）**：反复做四件事——造随机噪声；按第 9 章公式加噪；让网络猜噪声并算损失；梯度下降走一小步。这就是第 6 章“蒙眼下山”的真实代码版。

9. 第 44–51 行 (等变性检验): 把输入整体转 90° , 看输出是不是恰好也转了 90° 。如果是, 等变性误差会小到接近 0 (浮点精度量级, 通常小于 10 的负 5 次方)。

想一想

如果把第 25 行改成**直接让网络输出绝对坐标** (比如 `return self.net(x)`), 等变性检验还会通过吗? 为什么?

(提示: 绝对坐标依赖于“你把分子摆在什么地方”, 而相对位移不依赖。)

如果你没有 PyTorch 环境怎么办

你可以用纸笔复现这段代码的核心: 拿第 11 章的三个点, 手工算出相对位移矩阵 ($3 \times 3 \times 2$) 和距离矩阵 (3×3), 然后随便给一组权重, 按第 25 行的公式加权求和——得到的就是“等变输出”。**数学不在代码里, 在你手上。**

本篇小结 (你已会什么)

- 用三个点手算完了全文的全部数学: 距离矩阵与三角不等式、旋转等变性、扩散一步去噪;
- 理解了“**不变权重 $\times$ 等变位移 = 等变输出**”这一行代码背后的几何原理;
- 读懂了一份能跑的 PyTorch 参考实现, 包括训练循环与等变性检验。

下一篇: 全景对比、避坑指南、名词表与自测题, 以及给进阶读者的严格数学。

第七部分 第七篇 · 全景、避坑与附录

14 第 13 章 全景对比、常见误解与避坑指南

14.1 代际演进对比表

14.2 常见误解 Q&A

这一部分专门澄清初学者最容易踩的认知坑。

误解 1: “pLDDT 低 = 模型预测错了”

不一定。pLDDT 低常常意味着这段蛋白质**本来就没有固定形状** (天然无序区, IDR)。它们像一根软绳不停甩动, 在细胞中往往负责信号传递等“需要灵活”的功能。**模型给出低分, 恰恰是正确认识**。当然, pLDDT 低也可能是真错了, 所以要结合 PAE 与生物学知识一起判断。

误解 2: “AlphaFold 预测的构象就是蛋白质在细胞里的真实样子”

不一定。AlphaFold 给出的是**最可能的一种静态构象**。而真实的蛋白质是**会动的**: 它可能有多个构象 (结合配体前后不同)、会在不同 pH / 盐浓度 / 温度下改变形状、还会被磷酸化等修饰改变行为。它是“一张快照”, 不是“一段视频”。

表 1: 蛋白质结构预测与设计方法的代际演进对比

方法（年份）	类型	核心思路	关键局限
同源建模 (1990s-)	预测	套用同源模板	无模板即失效，“黄昏区”崩
分子动力学 (MD)	预测	牛顿力学逐步积分	力场近似、时间尺度差几个数量级
Rosetta (1998-)	预测/设计	能量函数 + 蒙特卡洛搜索	打分不精确、需海量采样、慢
AlphaFold1 (2018)	预测	预测距离图 + 势能优化	几何可能不自洽、MSA 依赖、不能设计
AlphaFold2 (2020)	预测	双轨 + 三角注意力 + IPA	单体为主、孤儿蛋白弱、不能设计
RoseTTAFold (2021)	预测	三轨网络	精度略逊于 AF2
ESMFold (2022)	预测	蛋白质语言模型取代 MSA	速度快、孤儿蛋白友好，精度略降
AlphaFold3 (2024)	预测	Pairformer + 扩散模块	复合物强，计算开销更大
ProteinMPNN (2022)	设计	反向折叠（结构 → 序列）	依赖给定骨架的质量
RFdiffusion (2023)	设计	SE(3) 上去噪扩散	必须做自洽性检验与湿实验

误解 3：“AI 设计出的蛋白质，计算机里折得好 = 一定做得出、能用”

差得远。计算机里的“成功”只说明这条设计**自洽**。到了湿实验，它还可能失败于：表达不出来（细菌/细胞不生产它）、不可溶（聚成一团沉淀）、不稳定（室温放两天就坏）、有免疫原性（注入体内被免疫系统攻击）。**自洽性检验是必要但不充分的筛子。**

误解 4：“AI 已经解决了生物学”

没有。AlphaFold 解决了“序列 → 结构”，但生物学还有一大串更难的问题：**动态过程**（蛋白质怎么动）、**相互作用网络**（谁和谁在什么时候结合）、**功能预测**（这个蛋白到底干什么）、**细胞与组织尺度**（从分子到生命）。**结构只是其中一环。**

14.3 工程落地避坑指南

1. **先读置信度，再读结构。**看 pLDDT（逐残基可靠性）与 PAE（结构域之间相对位置的可靠性）。做复合物时 PAE 往往比 pLDDT 更关键。
2. **孤儿蛋白换路线。**找不到同源序列时，MSA 信号弱，可改用蛋白质语言模型路线（ESM-Fold），但要接受一定的精度折让。
3. **抗体要小心。**抗体的 CDR 环（负责识别抗原的那几段）本身**高度可变**，正是演化上“故意多变”的区域，预测难度天然更高，需要专门微调与更多采样。
4. **警惕分布外。**模型在 PDB 里见过的折叠类型上表现极好，但对**全新折叠**可能给出“看起来很自信的错误结构”。对新靶点要保持怀疑，用多种证据交叉验证。
5. **复合物要注意化学细节。**涉及小分子配体时，**手性、质子化状态、电荷**这些输入细节一旦填错，输出会完全跑偏。
6. **算力与流程成本。**MSA 搜索（JackHMMER / HHblits）和模型推理都是瓶颈。批量任务要**预建 MSA 缓存**；设计流程中 ProteinMPNN 的采样条数直接决定候选池大小，需要在“多样性”和“实验预算”之间权衡。

14.4 这个领域还剩下的开放问题

- **动态与构象系综：**如何预测蛋白质“会怎么动”，而不只是一个静态结构？
- **真正可靠的从头设计：**让设计成功率从“筛很多个才中一个”提升到“设计即所得”。
- **孤儿蛋白与全新折叠：**摆脱对进化线索（MSA）的依赖。
- **从结构到功能：**知道了形状，能自动推断它干什么活吗？
- **干湿闭环自动化：**把“设计 → 实验 → 反馈 → 再设计”做成全自动机器人科学家。

14.5 论文与评测导航

- **CASP** (Critical Assessment of protein Structure Prediction): 两年一度的结构预测“奥运会”, AlphaFold 的成名地。核心指标 GDT_TS、TM-score。
- **CAFA**: 蛋白质功能预测评测, 与结构预测互补。
- **Nature / Science / Nature Methods / Nature Structural & Molecular Biology**: AlphaFold2 (Nature 2021)、AlphaFold3 (Nature 2024)、Rosetta 与 ProteinMPNN (Science 2022)、RFdiffusion (Nature 2023)。
- **NeurIPS / ICML / ICLR**: 等变深度学习 (SE(3)-Transformer、EGNN、Tensor Field Networks) 与 ML4Bio / MLSB / AI4Science Workshop。
- **入门练习**: AlphaFold 有公开的 Colab 笔记本, ProteinMPNN 与 RFdiffusion 也有开源实现, 可以直接在浏览器里跑通本文的设计闭环。

A 附录 A 阶梯 $0 \rightarrow 1 \rightarrow 2$: 给想深入的读者

这一节把全文的数学底座按三个阶梯重新整理: 阶梯 0 是直觉, 阶梯 1 是手算, 阶梯 2 是严格表述。如果前面的正文你已经读懂, 阶梯 2 可以慢慢来。

A.1 阶梯 0: 三句话的直觉

1. 蛋白质是一串珠子, 顺序决定形状 (Anfinsen);
2. 形状必须几何合法 (三角不等式) 且不随摆放方式改变判断 (等变性);
3. 设计就是从混沌噪声里把形状凿出来 (扩散), 再反推该用哪些珠子 (反向折叠)。

A.2 阶梯 1: 低维手算的闭环

第 11 章已经完整算过, 这里只列结果备查:

- 距离: $d_{12} = 3, d_{13} = 4, d_{23} = 5$, 满足 $1 \leq 5 \leq 7$;
- 旋转 90° 后: 坐标变为 $(0, 0), (0, 3), (-4, 0)$, 距离不变;
- 扩散: 加噪得 $\mathbf{x}_t = (0.0894, 3.4435)$, 损失 0.0008, 反推 $\hat{\mathbf{x}}_0 = (0.0100, 3.9900)$, 误差约 0.0141。

A.3 阶梯 2: 严格数学表述

A.3.1 群、不变性与等变性

设 G 是变换群 (蛋白质场景取 $SE(3)$, 即三维刚体运动群), $g \in G$ 作用在结构 $\mathbf{x}$ 上。称网络 f :

$$\text{等变 (equivariant)} \iff f(g \cdot \mathbf{x}) = g \cdot f(\mathbf{x}), \quad (34)$$

$$\text{不变 (invariant)} \iff f(g \cdot \mathbf{x}) = f(\mathbf{x}). \quad (35)$$

IPA 的实现机制: 设 q_i, k_j 是在局部框架下计算的特征, 注意力权重取

$$\alpha_{ij} = \text{softmax}_j \left(w_q q_i^\top k_j - \gamma \|T_i \circ \mathbf{x}_i - T_j \circ \mathbf{x}_j\|^2 + b \right), \quad (36)$$

其中第二项是在局部框架 T_i 下计算的平方距离, 是不变量, 故 α_{ij} 不变; 再令输出为 $\sum_j \alpha_{ij} (T_i^{-1} \circ \mathbf{x}_j - \mathbf{x}_i)$, 即“不变权重 $\times$ 等变位移”, 输出即为等变。

为什么等变性至关重要: 不等变的网络必须把同一个分子的所有旋转版本都当成新样本重新学习, 数据效率极低; 等变网络把对称性写进架构, 等价于免费的无穷数据增广。在结构数据极其稀缺 (PDB 仅约二十万) 的现实下, 这是生死线。

A.3.2 三角更新与三角不等式

Evoformer 的三角更新在 Pair 表示上强制“两条路径一致”: 从 i 到 j 的关系, 应与“经 k 中转”一致。写成两个方向:

$$\mathbf{z}_{ij} \leftarrow \mathbf{z}_{ij} + \sum_k a_{ik} \mathbf{z}_{ik}^\top \mathbf{b}_{kj}, \quad (37)$$

$$\mathbf{z}_{ij} \leftarrow \mathbf{z}_{ij} + \sum_k a_{jk} \mathbf{z}_{jk}^\top \mathbf{b}_{ki}. \quad (38)$$

这与三条边必须满足三角不等式是同一种约束的神经网络表达——它让 Pair 表示始终像一个真实三维结构的距离与朝向关系, 从源头避免 AlphaFold1 的几何不一致。

A.3.3 在 SE(3) 流形上扩散

把每个残基表示为一个刚体框架 $T = (\mathbf{R}, \mathbf{t}) \in \text{SE}(3)$, $\mathbf{R} \in \text{SO}(3)$ 是朝向, $\mathbf{t} \in \mathbb{R}^3$ 是平移。扩散需在两个不同的空间分别定义:

- **平移** $\mathbf{t} \in \mathbb{R}^3$: 欧氏空间, 直接加高斯噪声, 与图像/视频扩散完全一致;
- **旋转** $\mathbf{R} \in \text{SO}(3)$: 弯曲的流形, 不能简单加高斯噪声, 需在 $\text{SO}(3)$ 上定义随机扰动 (如绕随机轴转一个随噪声增大的角度), 并用黎曼几何视角的分数指导去噪。

训练目标仍可写成预测噪声/分数, 只是分数被分解为平移分量与旋转分量。

与系列其它文章的统一: 本系列 `vla_flow_matching` (连续动作流匹配 VLA) 与 `world_model_generation` (视频生成即世界模型) 用的也是“扩散/流匹配”范式。三者是同一套生成模型语言, 只是数据的几何不同: 动作空间、视频潜空间、SE(3) 流形。

A.3.4 反向折叠的自回归分解

ProteinMPNN 求解 $p(\text{序列} \mid \text{结构})$, 按残基自回归分解:

$$p(s_{1:N} \mid \mathbf{x}) = \prod_{i=1}^N p(s_i \mid \mathbf{x}, s_{<i}), \quad (39)$$

每个条件分布由一个结构到序列的等变图网络给出。由于序列简并性 (多个序列折成同一骨架), 该分布高度多峰——采样多条即得多个候选, 再交 AlphaFold 筛选。

A.3.5 评估指标速览

- **RMSD**: 预测与实验结构的原子坐标均方根偏差, 越小越好, 但对局部错误过于敏感;
- **GDT_TS**: 多个距离阈值下“命中原子比例”的平均, CASP 核心指标 (0–100);
- **TM-score**: 对长度归一化、对全局拓扑更鲁棒 (> 0.5 通常认为折叠正确);
- **lDDT / pLDDT**: 无需参考结构的局部距离差异检验, AlphaFold 用它给出逐残基置信度 (0–100)。

B 附录 B 名词速查表

名词	一句话解释
氨基酸	构成蛋白质的“珠子”, 常用 20 种
一级/二级/三级结构	序列 / 局部标准件 (螺旋、折叠) / 整体三维形状
Anfinsen 法则	序列本身就包含了结构的全部信息
莱文索尔悖论	随机试遍所有构象需超宇宙年龄, 实际只需毫秒到秒
折叠漏斗	能量景观是漏斗, 顺坡下滑即可抵达天然构象
MSA (多序列比对)	把同源序列对齐排好, 用来找共进化线索
共进化	两个位置总是一起变, 暗示它们在空间上相互接触
孤儿蛋白	找不到任何同源序列的蛋白, MSA 为空
距离矩阵	记录两两距离的表, 沿对角线对称
三角不等式	两边之和 $\geq$ 第三边; 是结构合法性的关卡
不变性	输入旋转平移后, 输出的值不变 (如距离)
等变性	输入旋转平移后, 输出跟着同样变换 (如坐标)
SE(3) / SO(3)	三维刚体运动群 / 三维旋转群 (后者是弯曲空间)
损失函数	给模型打分的“差多少”
梯度下降	蒙眼下山: 沿坡度向下走一小步
注意力 (Q/K/V)	用 Q 匹配 K 得到权重, 再加权平均 V
扩散模型	学会“把加进去的噪声猜出来并减掉”, 从而从噪声生成新对象
反向折叠	结构 $\rightarrow$ 序列 (ProteinMPNN)
从头设计	无中生有生成新骨架 (RFdiffusion)
自洽性检验	把设计出的序列丢回 AlphaFold 重折, 看能否折回目标结构
pLDDT / PAE	逐残基置信度 / 残基对相对位置误差预测
GDT_TS / TM-score	CASP 的结构相似度评分指标

C 附录 C 自测题与答案

自测题

1. 已知两点 $A(1, 2, 3)$ 与 $B(4, 6, 3)$ ，求距离。
2. 三边长度为 2, 3, 6，能构成三角形吗？为什么？
3. 把点 $(2, 0)$ 绕原点逆时针旋转 90° ，新坐标是多少？它到原点的距离变了吗？
4. 扩散模型的训练目标是什么？用一句话说。
5. 为什么“等变性”对蛋白质结构模型特别重要？
6. 一条 100 个氨基酸的链，若每个残基有 3 种姿势，构象总数约是多少？
7. 设计闭环里，“自洽性检验”这一步在做什么？为什么能省钱？

参考答案

1. $d = \sqrt{(1-4)^2 + (2-6)^2 + (3-3)^2} = \sqrt{9+16+0} = 5$ 。
2. **不能**。因为 $2+3=5 < 6$ ，违反三角不等式（两边之和必须大于第三边）。
3. 新坐标是 $(0, 2)$ 。到原点的距离**不变**，仍为 2（这正是不变性）。
4. 让网络猜“当初加进去的噪声是什么”，损失就是猜噪声值与真实噪声值的平方误差。
5. 因为同一个分子有无数种摆放朝向；等变网络只需学一次，等价于免费获得无穷的数据增广，在结构数据稀缺时至关重要。
6. $3^{100} \approx 5 \times 10^{47}$ 。
7. 把设计出的序列重新丢给 AlphaFold 预测，看它折回来的结构是否与设计目标吻合。它能用几秒到几分钟的**纯计算**，筛掉绝大多数不靠谱候选，从而把昂贵耗时的湿实验留给最有希望的少数几条。